

CodroidCS SDK 手册

版本: 2.1.3 | 命名空间: Codroid

目录

#	章节	说明
1	快速上手	安装、连接并运行第一个程序
2	核心概念	生命周期、TCP 模型、单位约定、异常处理
3	CodroidClient API	CodroidClient 完整 API 参考
4	运动 API	JointPoint、CartesianPoint、MoveInstruction、MotionWaitOptions、枚举
5	数据类型与枚举	CommonResponse、CriRealTimeData、RobotFrame、RegisterReadValue、异常
6	CRI 实时数据与控制	CriRealtimeDispatcher、TrajectoryGenerator、TrajectoryRequest、PacketParser
7	IO 与寄存器	DI/DO/AI/AO 操作、寄存器读写
8	辅助工具	发布/订阅、全局变量、运动学、ConsoleUtf8
9	.NET Framework 4.6.2 说明	net462 平台限制、250Hz SLA、兼容层

环境要求

目标框架	平台	说明
net6.0	Linux、Windows、macOS	.NET 6 SDK
net8.0	Linux、Windows、macOS	.NET 8 SDK (推荐)
net462	仅 Windows	.NET Framework 4.6.2+, 兼容 WinForms/WPF

通过 NuGet 安装

```
dotnet add package Codroidsdk
```

项目引用

```
dotnet add path/to/YourApp.csproj reference path/to/CodroidSDK/CodroidCS.csproj
```

API 命名约定

所有公共方法返回 `Task` / `Task<T>`，但不使用 `Async` 后缀。

```
// 正确
await robot.ConnectRemoteAndSwitchOn();
int di = await robot.GetDi(0);

// 错误
await robot.ConnectRemoteAndSwitchOnAsync(); // 不存在
```

这样做是为了让 C# / Python / C++ 三套 SDK 使用同一套公开函数名。

单位约定

层级	线性	角度
SDK 公共 API	mm	deg（度）
TCP JSON 协议	mm	deg
CRI UDP 二进制（线路层）	m	rad（弧度）
<code>CriRealTimeData</code> （已解析）	mm	deg

`CriRealtimePacketParser.Parse()` 和 `CriRealtimeDispatcher`（`convertToSi=true`）会自动处理 m 与 mm、rad 与 deg 的换算。

快速上手

安装

NuGet

```
dotnet add package Codroidsdk
```

项目引用

```
dotnet add path/to/YourApp.csproj reference path/to/CodroidSDK/CodroidCS.csproj
```

支持的目标框架

```
<!-- net8.0 (推荐) -->
<TargetFramework>net8.0</TargetFramework>

<!-- net6.0 -->
<TargetFramework>net6.0</TargetFramework>

<!-- .NET Framework 4.6.2+ (仅 Windows) -->
<TargetFramework>net462</TargetFramework>
```

最小示例

连接控制器，读取数字输入，写入数字输出，然后断开。

```
using Codroid;

var robot = new CodroidClient("192.168.8.136");

try
{
    // 连接、切换远程、上电
    await robot.ConnectRemoteAndSwitchOn();
    // ⚠️ 启动 CRI 数据推送（阻塞运动必需）
    await robot.StartCriDataPush("192.168.8.150", 18888);
    await robot.WaitForCriData(5.0); // 等待首帧

    // 读取 DI 端口 0
    int di0 = await robot.GetDi(0);
    Console.WriteLine($"DI 0 = {di0}");

    // 将 DI 值写入 DO 端口 10
    await robot.SetDo(10, di0);
}
finally
{
}
```

```
// 始终在 finally 中断开
robot.Disconnect();
}
```

完整工作流示例

```
using Codroid;

ConsoleUtf8.InitConsoleUtf8(); // Windows 控制台 UTF-8

var robot = new CodroidClient("192.168.8.136");

try
{
    // 1. 连接
    await robot.ConnectRemoteAndSwitchOn();
    // ⚠️ 启动 CRI 数据推送（阻塞运动必需）
    await robot.StartCriDataPush("192.168.8.150", 18888);
    await robot.WaitForCriData(5.0); // 等待首帧

    // 2. IO 操作
    int di0 = await robot.GetDi(0);
    await robot.SetDo(10, di0);

    // 3. 寄存器
    RegisterReadValue reg = await robot.GetRegisterValue(49100);
    int value = reg.GetInt32();
    await robot.SetRegisterValue(49100, value + 1);

    // 4. 运动
    await robot.MovJ(JointPoint.Degrees(new[] { 0, 0, 90, 0, 90, 0 }), speed: 40, acc: 100);

    // 5. 阻塞运动
    robot.MovLSync(
        CartesianPoint.MmDegWithRef(new[] { 400, 0, 300, 180, 0, 0 },
robot.CriData.JointPosition),
        speed: 150, acc: 500);
}
finally
{
    robot.Disconnect();
}
```

运行示例项目

```
# net8.0 (完整套件)
dotnet run --project CodroidTestNet8/CodroidTestNet8.csproj

# 指定控制器 IP
dotnet run --project CodroidTestNet8/CodroidTestNet8.csproj -- 192.168.8.10

# 仅运行某一类演示
dotnet run --project CodroidTestNet8/CodroidTestNet8.csproj -- cri 192.168.8.10
dotnet run --project CodroidTestNet8/CodroidTestNet8.csproj -- io 192.168.8.10
dotnet run --project CodroidTestNet8/CodroidTestNet8.csproj -- register 192.168.8.10

# net462 / .NET Framework 4.6.2
dotnet run --project CodroidTestNet462/CodroidTestNet462.csproj -- 192.168.8.10
```

错误处理

所有 TCP 指令在失败时抛出异常：

异常	条件
CodroidCommandException	控制器返回 err
TimeoutException	10 秒内未收到响应
ArgumentException	参数无效 (SDK 侧校验)

```
try
{
    await robot.SetDo(999, 1); // 无效端口
}
catch (CodroidCommandException ex)
{
    Console.WriteLine($"控制器错误: {ex.ControllerError}");
}
catch (TimeoutException)
{
    Console.WriteLine("请求超时");
}
```

核心概念

CodroidClient 生命周期

```
new CodroidClient(ip)
    |
    v
Connect() --or-- ConnectRemoteAndSwitchOn()
    |
    v
[ IO / Register / Motion / CRI ... ]
    |
    v
Disconnect()
```

```
var robot = new CodroidClient("192.168.8.136");

try
{
    await robot.ConnectRemoteAndSwitchOn();
    // ... 使用 robot ...
}
finally
{
    robot.Disconnect(); // 始终在 finally 中调用
}
```

构造函数

```
var robot = new CodroidClient(string ip);
```

- `ip` -- 控制器 IP 地址
- TCP 端口固定为 **9001**

属性

属性	类型	说明
<code>CriData</code>	<code>CriRealTimeData</code>	CRI 数据快照的线程安全副本
<code>Data</code>	<code>CriRealTimeData</code>	内部 CRI 缓冲区的直接引用（更快，非线程安全）

事件

```
robot.CriDataReceived += data =>
{
    Console.WriteLine($"Joints: {string.Join(", ", data.JointPosition)}");
};
```

每当解析完一个有效的 CRI UDP 帧后触发。`data` 参数是线程安全的副本。

TCP 指令模型

每个与控制器通信的 SDK 方法都遵循以下模式：

- 1. SDK 分配唯一的 `id`
- 2. SDK 将 `{ id, ty, db }` 序列化为 JSON 并通过 TCP 发送
- 3. 控制器响应 `{ id, ty, db, err }`
- 4. SDK 通过 `id` 匹配响应
- 5. 若 `err` 非空则抛出 `CodroidCommandException`
- 6. 若 10 秒内未收到响应则抛出 `TimeoutException`

CommonResponse

```
public class CommonResponse
{
    public object? id { get; set; } // 请求 ID
    public string? ty { get; set; } // 响应类型
    public JsonElement db { get; set; } // 业务数据
    public string? err { get; set; } // 错误信息
}
```

大多数方法返回 `Task<CommonResponse>`。`db` 字段包含实际结果数据。

单位约定

SDK 公共 API 使用毫米和度。这与 TCP JSON 协议一致。

上下文	线性	角度
SDK API、TCP JSON	mm	deg
CRI UDP 线路格式	m	rad
<code>CriRealTimeData</code> （已解析）	mm	deg

重要: CRI UDP 二进制载荷使用米和弧度。SDK 在 `CriRealtimePacketParser.Parse()` 和 `CriRealtimeDispatcher`（`convertToSi=true`）中自动转换为 mm/deg。不要假设原始 UDP 浮点数是 mm/deg。

异步命名约定

所有公共方法返回 `Task` 或 `Task<T>`，但**不**使用 `Async` 后缀。

```
// 这些是异步方法 -- 需要 await
await robot.ConnectRemoteAndSwitchOn();
int di = await robot.GetDi(0);
await robot.MovJ(JointPoint.Degrees(joints), 40, 100);
```

这样设计是为了让 C# / Python / C++ 三套 SDK 使用同一套公开函数名。

异常类型

异常	触发条件	来源
<code>CodroidCommandException</code>	控制器返回 <code>err</code> 字段	TCP 响应
<code>TimeoutException</code>	10 秒内未收到响应	TCP 等待
<code>ArgumentException</code>	参数值无效	SDK 校验
<code>ArgumentOutOfRangeException</code>	参数超出范围（如 DO 端口）	SDK 校验
<code>InvalidOperationException</code>	未连接	SDK 状态
<code>ObjectDisposedException</code>	对象已释放	SDK 状态

CodroidCommandException 属性

```
public class CodroidCommandException : Exception
{
    public int RequestId { get; } // 协议请求 ID
    public string CommandType { get; } // 如 "Robot/move"
    public string? ControllerError { get; } // 控制器的 err 字段
    public CommonResponse? Response { get; } // 完整响应
}
```

线程安全

- `CriData` -- 线程安全（返回副本）
- `Data` -- 非线程安全（直接引用）
- 所有 TCP 方法 -- 可从任意线程调用，但不要在同一 `CodroidClient` 上并发调用
- `CriRealtimeDispatcher` -- `SendCommand` / `SendTrajectory` 非线程安全

CodroidClient API 参考

类: CodroidClient
命名空间: Codroid
源文件: CodroidSDK/Codroid.cs

构造函数

```
public CodroidClient(string ip)
```

参数	类型	说明
ip	string	控制器 IP 地址

TCP 端口固定为 9001。

```
var robot = new CodroidClient("192.168.8.136");
```

属性

属性	类型	说明
CriData	CriRealTimeData	CRI 数据快照的线程安全副本
Data	CriRealTimeData	内部 CRI 缓冲区直接引用（更快，非线程安全）

```
// 线程安全（返回副本）
double[] joints = robot.CriData.JointPosition;

// 直接引用（更快）
double[] joints2 = robot.Data.JointPosition;
```

事件

```
public event Action<CriRealTimeData>? CriDataReceived
```

每当解析完一个有效的 CRI UDP 帧后触发。参数是线程安全的副本。

```
robot.CriDataReceived += data =>
{
    Console.WriteLine($"Joints: {string.Join(", ", data.JointPosition)}");
    Console.WriteLine($"TCP: {string.Join(", ", data.TcpPose)}");
    Console.WriteLine($"InMotion: {data.InMotion}");
};
```

1. 连接管理

Connect

```
public async Task Connect()
```

建立与控制器的 TCP 连接。

```
await robot.Connect();
```

ConnectRemoteAndSwitchOn

```
public async Task ConnectRemoteAndSwitchOn()
```

连接 TCP、经 auto 切换远程模式、然后上电。推荐的一键初始化方法。

```
await robot.ConnectRemoteAndSwitchOn();
```

Disconnect

```
public void Disconnect()
```

停止 CRI UDP 监听并断开 TCP。始终在 `finally` 中调用。

```
try
{
    await robot.ConnectRemoteAndSwitchOn();
    // ... 操作 ...
}
finally
{
    robot.Disconnect();
}
```

2. 模式切换

SwitchOn / SwitchOff

```
public async Task<CommonResponse> SwitchOn()
public async Task<CommonResponse> SwitchOff()
```

机器人上电 / 下电。

方法	协议指令
<code>SwitchOn()</code>	<code>Robot/switchOn</code>
<code>SwitchOff()</code>	<code>Robot/switchOff</code>

```
await robot.SwitchOn();
// ... 操作 ...
await robot.SwitchOff();
```

ToManual / ToAuto / ToRemote

```
public Task<CommonResponse> ToManual()
public Task<CommonResponse> ToAuto()
public Task<CommonResponse> ToRemote()
```

切换到手动 / 自动 / 远程模式。需要固件 2.3.2.6+。

方法	协议指令
<code>ToManual()</code>	<code>Robot/toManual</code>
<code>ToAuto()</code>	<code>Robot/toAuto</code>
<code>ToRemote()</code>	<code>Robot/toRemote</code>

EnterManualModeViaAuto / EnterRemoteModeViaAuto

```
public async Task<CommonResponse> EnterManualModeViaAuto()
public async Task<CommonResponse> EnterRemoteModeViaAuto()
```

先切换到自动，再切换到手动/远程。满足控制器"必须经过自动模式"的限制。

```
await robot.EnterRemoteModeViaAuto();
```

ToSimulation / ToActual

```
public Task<CommonResponse> ToSimulation()
public Task<CommonResponse> ToActual()
```

切换到仿真 / 实机模式。

StartDrag / StopDrag

```
public Task<CommonResponse> StartDrag()  
public Task<CommonResponse> StopDrag()
```

进入 / 退出拖拽模式。需要固件 2.3.2.6+。

ClearSystemError

```
public Task<CommonResponse> ClearSystemError()
```

清除系统错误状态。

方法	协议指令
ClearSystemError()	System/clearError

3. 运动指令（非阻塞）

所有运动方法发送指令后立即返回。使用 *Sync 变体进行阻塞等待。

MovJ -- 关节运动

```
public Task<CommonResponse> MovJ(JointPoint target, double speed, double acc,  
    double blend = 25, double[]? coor = null, double[]? tool = null, double? relativeBlend =  
    null)  
  
public Task<CommonResponse> MovJ(CartesianPoint target, double speed, double acc,  
    double blend = 25, double[]? coor = null, double[]? tool = null, double? relativeBlend =  
    null)
```

参数	类型	默认值	说明
target	JointPoint / CartesianPoint	--	目标位置
speed	double	--	速度
acc	double	--	加速度
blend	double	25	平滑半径
coor	double[]?	null	用户坐标系
tool	double[]?	null	工具坐标系
relativeBlend	double?	null	相对平滑

```
// 关目标
await robot.MovJ(JointPoint.Degrees(new[] { 0, 0, 90, 0, 90, 0 })), speed: 40, acc: 100);

// 笛卡尔目标（关节运动到 TCP 位姿）
await robot.MovJ(CartesianPoint.MmDeg(new[] { 400, 0, 300, 180, 0, 0 })), speed: 40, acc: 100);
```

MovL -- 直线运动

```
public Task<CommonResponse> MovL(CartesianPoint target, double speed, double acc,
    double blend = 25, double[]? coor = null, double[]? tool = null, double? relativeBlend = null)

public Task<CommonResponse> MovL(JointPoint target, double speed, double acc,
    double blend = 25, double[]? coor = null, double[]? tool = null, double? relativeBlend = null)
```

```
// 笛卡尔直线运动
await robot.MovL(CartesianPoint.MmDegWithRef(pose, robot.CriData.JointPosition),
    speed: 150, acc: 500);

// 直线运动到关节目标
await robot.MovL(JointPoint.Degrees(new[] { 10, 20, 90, 0, 90, 0 })), speed: 100, acc: 300);
```

MovC -- 圆弧运动

```
public Task<CommonResponse> MovC(CartesianPoint middle, CartesianPoint target,
    double speed, double acc, double blend = 25,
    double[]? coor = null, double[]? tool = null, double? relativeBlend = null)
```

参数	类型	说明
middle	CartesianPoint	中间点（圆弧上）
target	CartesianPoint	终点

```
await robot.MovC(
    CartesianPoint.MmDeg(new[] { 450, 100, 300, 180, 0, 0 })),
    CartesianPoint.MmDeg(new[] { 500, 0, 300, 180, 0, 0 })),
    speed: 100, acc: 300);
```

MovCircle -- 整圆运动

```
public Task<CommonResponse> MovCircle(CartesianPoint middle, CartesianPoint target,
    int circleNum, double speed, double acc, double blend = 25,
    double[]? coor = null, double[]? tool = null, double? relativeBlend = null)
```

参数	类型	说明
middle	CartesianPoint	中间点
target	CartesianPoint	终点
circleNum	int	整圆圈数

```
await robot.MovCircle(
    CartesianPoint.MmDeg(mid),
    CartesianPoint.MmDeg(end),
    circleNum: 1, speed: 80, acc: 200);
```

Move -- 多段路径

```
public async Task<CommonResponse> Move(IReadOnlyList<MoveInstruction> instructions)
```

将一组运动指令作为单条路径命令发送。

```
await robot.Move(new[]
{
    MoveInstruction.MovJ(JointPoint.Degrees(new[] { 0, 0, 90, 0, 90, 0 }), 40, 100),
    MoveInstruction.MovL(CartesianPoint.MmDegWithRef(pose, robot.CriData.JointPosition),
    150, 500),
    MoveInstruction.MovC(CartesianPoint.MmDeg(mid), CartesianPoint.MmDeg(end), 100, 300),
});
```

4. 阻塞运动指令

`*Sync` 方法发送运动指令后阻塞直到 CRI 确认机器人到达目标。成功返回 `true`，错误/超时抛出异常。

必须先调用 `StartCriDataPush`。

MovJSync

```
public bool MovJSync(JointPoint target, double speed, double acc,
    MotionWaitOptions? wait = null, double blend = 25,
    double[]? coor = null, double[]? tool = null, double? relativeBlend = null)

public bool MovJSync(CartesianPoint target, double speed, double acc,
    MotionWaitOptions? wait = null, double blend = 25,
    double[]? coor = null, double[]? tool = null, double? relativeBlend = null)
```

```
var wait = new MotionWaitOptions
{
    Timeout = TimeSpan.FromSeconds(90),
    JointToleranceDeg = 0.3
};

robot.MovJSync(JointPoint.Degrees(new[] { 0, 0, 90, 0, 90, 0 }), 40, 100, wait);
```

MovLSync

```
public bool MovLSync(CartesianPoint target, double speed, double acc,
    MotionWaitOptions? wait = null, double blend = 25,
    double[]? coor = null, double[]? tool = null, double? relativeBlend = null)

public bool MovLSync(JointPoint target, double speed, double acc,
    MotionWaitOptions? wait = null, double blend = 25,
    double[]? coor = null, double[]? tool = null, double? relativeBlend = null)
```

```
robot.MovLSync(
    CartesianPoint.MmDegWithRef(pose, robot.CriData.JointPosition),
    speed: 150, acc: 500,
    wait: new MotionWaitOptions
    {
        Timeout = TimeSpan.FromSeconds(60),
        CartesianPositionToleranceMm = 2.0,
        CartesianOrientationToleranceDeg = 1.5
    });
```

MovCSync

```
public bool MovCSync(CartesianPoint middle, CartesianPoint target,
    double speed, double acc, MotionWaitOptions? wait = null,
    double blend = 25, double[]? coor = null, double[]? tool = null, double? relativeBlend =
    null)
```

```
robot.MovCSync(
    CartesianPoint.MmDeg(mid), CartesianPoint.MmDeg(end),
    speed: 100, acc: 300);
```

MovCircleSync

```
public bool MovCircleSync(CartesianPoint middle, CartesianPoint target,
    int circleNum, double speed, double acc, MotionWaitOptions? wait = null,
    double blend = 25, double[]? coor = null, double[]? tool = null, double? relativeBlend =
    null)
```

MoveSync

```
public bool MoveSync(IReadOnlyList<MoveInstruction> instructions, MotionWaitOptions? wait = null)
```

发送多段路径并阻塞直到最后一段目标到达。

```
robot.MoveSync(new[]
{
    MoveInstruction.MovJ(JointPoint.Degrees(j1), 40, 100),
    MoveInstruction.MovL(CartesianPoint.MmDegWithRef(p2, refJ), 150, 500),
});
```

5. 运动控制

PauseRobotMotion

```
public async Task<CommonResponse> PauseRobotMotion()
```

暂停当前运动。协议指令：`Robot/pause`。

ResumeRobotMotion

```
public async Task<CommonResponse> ResumeRobotMotion()
```

恢复暂停的运动。协议指令：`Robot/resume`。

StopRobotMove

```
public async Task<CommonResponse> StopRobotMove()
```

立即停止当前运动。协议指令：`Robot/stopMove`。

6. MoveTo 指令

MoveTo

```
public async Task<CommonResponse> MoveTo(MoveToKind kind, MoveToTarget? target = null)
```

移动到预设或规划位置。运行期间需要心跳。

方法	协议指令
<code>MoveTo(kind, target)</code>	<code>Robot/moveTo</code>

```
// 移动到原点
await robot.MoveTo(MoveToKind.Home);

// 移动到安全位
await robot.MoveTo(MoveToKind.Safe);

// 移动到指定关节位置
await robot.MoveTo(MoveToKind.JointPlanned, MoveToTarget.Joint(JointPoint.Degrees(joints)));
```

MoveToHeartbeat

```
public async Task<CommonResponse> MoveToHeartbeat()
```

发送心跳以维持 MoveTo 运动。约每 500ms 调用一次。

StopMoveTo

```
public async Task<CommonResponse> StopMoveTo()
```

停止当前 MoveTo / RunTo 运动。

7. Jog 指令

StartJog

```
public async Task<CommonResponse> StartJog(RobotJogParameters parameters)
```

启动 Jog。需要约每 500ms 发送心跳。

```
var jogParams = RobotJogParameters.Create(
    RobotJogMode.Joint,    // 关节模式
    speed: 10,             // 速度
    index: 0,              // 轴索引 (0-5)
    RobotJogFrameType.User, // 用户坐标系
    coorId: 0              // 坐标系 ID
);

await robot.StartJog(jogParams);

// 持续发送心跳
while (jogging)
{
    await Task.Delay(500);
    await robot.JogHeartbeat();
}

await robot.StopJog();
```

StopJog

```
public async Task<CommonResponse> StopJog()
```

停止 Jog。协议指令：`Robot/stopJog`。

JogHeartbeat

```
public async Task<CommonResponse> JogHeartbeat()
```

发送心跳以维持 Jog 状态。约每 500ms 调用一次。

8. IO 操作

GetDi / GetDo / GetAi / GetAo

```
public async Task<int> GetDi(int port)    // 读取数字输入
public async Task<int> GetDo(int port)    // 读取数字输出
public async Task<double> GetAi(int port) // 读取模拟输入
public async Task<double> GetAo(int port) // 读取模拟输出
```

方法	返回值	协议指令
<code>GetDi(port)</code>	<code>int</code> (0 或 1)	<code>IOManager/GetIOValue</code>
<code>GetDo(port)</code>	<code>int</code> (0 或 1)	<code>IOManager/GetIOValue</code>
<code>GetAi(port)</code>	<code>double</code>	<code>IOManager/GetIOValue</code>
<code>GetAo(port)</code>	<code>double</code>	<code>IOManager/GetIOValue</code>

```
int di0 = await robot.GetDi(0);
Console.WriteLine($"DI 0 = {di0}");

double ai1 = await robot.GetAi(1);
Console.WriteLine($"AI 1 = {ai1:F3}");
```

SetDo / SetAo

```
public async Task<CommonResponse> SetDo(int port, int value) // 写入 DO (0 或 1)
public async Task<CommonResponse> SetAo(int port, double value) // 写入 AO
```

方法	协议指令
<code>SetDo(port, value)</code>	<code>IOManager/SetIOValue</code>

方法	协议指令
<code>SetAo(port, value)</code>	<code>IOManager/SetIOValue</code>

```
await robot.SetDo(10, 1);    // 设置 DO 10 为高
await robot.SetDo(10, 0);    // 设置 DO 10 为低
await robot.SetAo(0, 3.14);  // 设置 AO 0 为 3.14
```

GetIoValues（批量读取）

```
public async Task<CommonResponse> GetIoValues(IReadOnlyList<(string Type, int Port)> pins)
```

```
var pins = new (string Type, int Port)[]
{
    ("DI", 0), ("DI", 1), ("DO", 10), ("AI", 0)
};

CommonResponse resp = await robot.GetIoValues(pins);
// 结果在 resp.db 中
```

9. 寄存器操作

GetRegisterValue

```
public async Task<RegisterReadValue> GetRegisterValue(int address)
```

```
RegisterReadValue reg = await robot.GetRegisterValue(49100);
int intVal = reg.GetInt32();
double dblVal = reg.GetDouble();
```

GetRegisterValues（批量读取）

```
public async Task<IReadOnlyList<RegisterReadValue>> GetRegisterValues(IReadOnlyList<int> addresses)
```

```
var addresses = new[] { 49100, 49101, 49102 };
IReadOnlyList<RegisterReadValue> values = await robot.GetRegisterValues(addresses);

for (int i = 0; i < values.Count; i++)
    Console.WriteLine($"Register {addresses[i]} = {values[i].GetInt32()}");
```

SetRegisterValue

```
public async Task<CommonResponse> SetRegisterValue(int address, int value)
public async Task<CommonResponse> SetRegisterValue(int address, double value)
```

```
await robot.SetRegisterValue(49100, 42);
await robot.SetRegisterValue(49101, 3.14);
```

SetExtendArrayType / RemoveExtendArray

```
public async Task<CommonResponse> SetExtendArrayType(int index, string type)
public async Task<CommonResponse> RemoveExtendArray(int index)
```

管理扩展数组元素（索引 0~999）。

```
await robot.SetExtendArrayType(0, RegisterExtendArrayValueType.Int32);
await robot.RemoveExtendArray(0);
```

10. 机器人设置（19.x 协议）

SetManualMoveRate / SetAutoMoveRate

```
public async Task<CommonResponse> SetManualMoveRate(int percent)
public async Task<CommonResponse> SetAutoMoveRate(int percent)
```

设置手动/自动运动倍率（1~100%）。

```
await robot.SetManualMoveRate(50); // 50% 速度
await robot.SetAutoMoveRate(100); // 全速
```

SetCollisionSensitivity

```
public async Task<CommonResponse> SetCollisionSensitivity(int sensitivity)
```

设置碰撞检测灵敏度（0~100）。固件 2.3.2.10+。

```
await robot.SetCollisionSensitivity(50);
```

SetPayload

```
public async Task<CommonResponse> SetPayload(int payloadId)
```

设置当前载荷槽位（0~15）。固件 2.3.2.10+。

```
await robot.SetPayload(1); // 使用载荷槽位 1
```

GetRobotParameters

```
public async Task<RobotParameters> GetRobotParameters()
```

获取所有设置界面参数（协议 19.7）。返回工具坐标系、载荷坐标系、用户坐标系及默认 ID。

```
RobotParameters param = await robot.GetRobotParameters();  
Console.WriteLine($"默认工具: {param.DefaultToolId}");  
Console.WriteLine($"默认载荷: {param.DefaultPayloadId}");  
Console.WriteLine($"最大载荷: {param.MaxPayload} kg");
```

SetDefaultPayloadId / SetDefaultToolId / SetDefaultUserCoordinateId

```
public Task<CommonResponse> SetDefaultPayloadId(int payloadId) // 0~15  
public Task<CommonResponse> SetDefaultToolId(int toolId) // 0~15  
public Task<CommonResponse> SetDefaultUserCoordinateId(int coordinateId) // 0~15
```

设置默认载荷/工具/用户坐标系槽位。

```
await robot.SetDefaultToolId(2);  
await robot.SetDefaultPayloadId(1);  
await robot.SetDefaultUserCoordinateId(0);
```

SaveToolFrames / SetToolFrame

```
public Task<CommonResponse> SaveToolFrames(IReadOnlyList<RobotFrame> frames)  
public async Task<CommonResponse> SetToolFrame(int frameId, RobotFrame frame)
```

保存完整工具坐标系表（必须包含 id 0~15，id=0 必须全零）/ 修改单个工具坐标系（先读后写，仅 id 1~15）。

```
// 设置单个工具坐标系  
await robot.SetToolFrame(1, new RobotFrame  
{  
    Id = 1, X = 0, Y = 0, Z = 100, A = 0, B = 0, C = 0  
});
```

SavePayloadFrames / SetPayloadFrame

```
public Task<CommonResponse> SavePayloadFrames(IReadOnlyList<RobotPayloadFrame> frames)
public async Task<CommonResponse> SetPayloadFrame(int frameId, RobotPayloadFrame frame)
```

保存完整载荷坐标系表 / 修改单个载荷坐标系 (id 1~15)。

```
await robot.SetPayloadFrame(1, new RobotPayloadFrame
{
    Id = 1, M = 2.5, Mx = 0, My = 0, Mz = 50
});
```

SaveUserCoordinateFrames / SetUserCoordinateFrame

```
public Task<CommonResponse> SaveUserCoordinateFrames(IReadOnlyList<RobotFrame> frames)
public async Task<CommonResponse> SetUserCoordinateFrame(int frameId, RobotFrame frame)
```

保存完整用户坐标系表 / 修改单个用户坐标系 (id 1~15)。

```
await robot.SetUserCoordinateFrame(1, new RobotFrame
{
    Id = 1, X = 100, Y = 200, Z = 0, A = 0, B = 0, C = 45
});
```

11. CRI 实时数据

StartCriDataPush

```
public async Task<CommonResponse> StartCriDataPush(string udpIp, int udpPort)
```

启动本地 UDP 监听并请求控制器推送 CRI 实时数据。固定参数：100ms 周期、高精度、mask 0xFFFF、308 字节 UDP 包。

方法	协议指令
StartCriDataPush(udpIp, udpPort)	CRI/StartDataPush

```
await robot.StartCriDataPush("192.168.8.150", 18888);

robot.CriDataReceived += data =>
{
    Console.WriteLine($"Joints: {string.Join(", ", data.JointPosition)}");
};
```

StopCriDataPush

```
public async Task<CommonResponse> StopCriDataPush(string? udpIp = null, int? udpPort = null)
```

请求控制器停止 CRI 数据推送并关闭本地 UDP 监听。

方法	协议指令
StopCriDataPush(ip, port)	CRI/StopDataPush

```
await robot.StopCriDataPush("192.168.8.150", 18888);
```

12. CRI 实时控制

StartCriControl

```
public async Task<CommonResponse> StartCriControl(int filterType = 1, int durationMs = 4, int startBuffer = 5)
```

启用 CRI 实时控制模式。

参数	类型	默认值	说明
filterType	int	1	0=关闭, 1=均值, 2=二阶低通, 3=椭圆
durationMs	int	4	控制周期 (1~16ms, 必须整除 1000)
startBuffer	int	5	起始缓冲帧数 (1~100)

方法	协议指令
StartCriControl(...)	CRI/StartControl

```
await robot.StartCriControl(filterType: 1, durationMs: 4, startBuffer: 5);
```

StopCriControl

```
public async Task<CommonResponse> StopCriControl()
```

禁用 CRI 实时控制模式。

方法	协议指令
StopCriControl()	CRI/StopControl

13. 项目执行

EnterRemoteScriptMode

```
public async Task<CommonResponse> EnterRemoteScriptMode()
```

请求进入远程脚本模式。

RunScript

```
public async Task<CommonResponse> RunScript(  
    string mainScript,  
    IReadOnlyDictionary<string, string>? subThreads = null,  
    IReadOnlyDictionary<string, string>? subPrograms = null,  
    IReadOnlyDictionary<string, string>? interrupts = null,  
    IReadOnlyDictionary<string, object>? vars = null)
```

发送脚本立即执行。

```
await robot.RunScript(mainScript: "movej(j1, v50) sub1() end");
```

Run / RunByIndex / RunStep

```
public async Task<CommonResponse> Run(string projectID)  
public async Task<CommonResponse> RunByIndex(int index)  
public async Task<CommonResponse> RunStep(string projectID)
```

按 ID / 索引启动项目 / 单步执行。

```
await robot.Run("project_001");  
await robot.RunByIndex(0);  
await robot.RunStep("project_001");
```

PauseProject / ResumeProject / StopProject

```
public async Task<CommonResponse> PauseProject()  
public async Task<CommonResponse> ResumeProject()  
public async Task<CommonResponse> StopProject()
```

方法	协议指令
PauseProject()	project/pause
ResumeProject()	project/resume
StopProject()	project/stop

14. 发布订阅

SubscribePublishTopic

```
public async Task<PublishTopicSubscription> SubscribePublishTopic(  
    string topicTy, Action<PublishNotification> handler, int tcMilliseconds = 100)
```

订阅 TCP 主题推送。返回可释放的订阅句柄。

```
using var sub = await robot.SubscribePublishTopic(  
    PublishTopics.RobotStatus,  
    notification =>  
    {  
        Console.WriteLine($"主题: {notification.Ty}");  
        Console.WriteLine($"数据: {notification.Db}");  
    });  
  
// 订阅在 sub.Dispose() 前有效  
await Task.Delay(10000);  
// 'using' 会自动调用 sub.Dispose()
```

15. 全局变量

GetGlobalVars / GetGlobalVarsCatalog

```
public async Task<CommonResponse> GetGlobalVars()  
public async Task<IReadOnlyDictionary<string, GlobalVarCatalogEntry>> GetGlobalVarsCatalog()
```

```
var catalog = await robot.GetGlobalVarsCatalog();  
foreach (var (name, entry) in catalog)  
{  
    Console.WriteLine($"{name} = {entry.Value} ({entry.Remark})");  
}
```

SaveGlobalVar / SaveGlobalVars

```
public Task<CommonResponse> SaveGlobalVar(string name, object value, string? remark = null)  
public async Task<CommonResponse> SaveGlobalVars(IReadOnlyCollection<GlobalVarSaveItem>  
items)
```

```
// 单个
await robot.SaveGlobalVar("counter", 42, "test counter");

// 批量
await robot.SaveGlobalVars(new[]
{
    new GlobalVarSaveItem("x", 100.0, "X position"),
    new GlobalVarSaveItem("y", 200.0, "Y position"),
});
```

RemoveGlobalVars

```
public async Task<CommonResponse> RemoveGlobalVars(IEnumerable<string> names)
```

删除指定全局变量。删除不存在的变量不会报错。

```
await robot.RemoveGlobalVars(new[] { "counter", "x", "y" });
```

16. 运动学

AposToCpos / AposToCposPose (正运动学)

```
public async Task<CommonResponse> AposToCpos(double[] jointDegrees, double[] userFrame,
double[] toolFrame, double[]? externalAxisPositions = null)
public async Task<double[]> AposToCposPose(double[] jointDegrees, double[] userFrame,
double[] toolFrame, double[]? externalAxisPositions = null)
```

正运动学：关节空间 -> 笛卡尔空间。`AposToCposPose` 返回 [x,y,z,rx,ry,rz]，单位 mm+deg。

```
double[] joints = { 0, 0, 90, 0, 90, 0 };
double[] userFrame = { 0, 0, 0, 0, 0, 0 };
double[] toolFrame = { 0, 0, 100, 0, 0, 0 };

double[] pose = await robot.AposToCposPose(joints, userFrame, toolFrame);
Console.WriteLine($"TCP: [{string.Join(", ", pose.Select(v => v.ToString("F1"))}")]");
// TCP: [400.0, 0.0, 300.0, 180.0, 0.0, 0.0]
```

CposToApos / CposToAposJoints (逆运动学)

```
public async Task<CommonResponse> CposToApos(double[] cartesianMmDeg, double[]
referenceJointDegrees, double[]? externalAxisPositions = null)
public async Task<double[]> CposToAposJoints(double[] cartesianMmDeg, double[]
referenceJointDegrees, double[]? externalAxisPositions = null)
```

逆运动学：笛卡尔 -> 关节空间。`CposToAposJoints` 返回 6 个关节角度（度）。

```
double[] pose = { 400, 0, 300, 180, 0, 0 };
double[] refJoints = { 0, 0, 90, 0, 90, 0 };

double[] joints = await robot.CposToAposJoints(pose, refJoints);
Console.WriteLine($"Joints: [{string.Join(", ", joints.Select(v => v.ToString("F2"))}]");
```

CalculateRelativePose / CalculateRelativePoseResult

```
public async Task<CommonResponse> CalculateRelativePose(double[] tcpPoseWorld, double[]
offset, RelativePoseCoorType coorType, double[]? tcpPoseInPosCoorFrame = null, double[]?
userCoorFrame = null)
public async Task<double[]> CalculateRelativePoseResult(double[] tcpPoseWorld, double[]
offset, RelativePoseCoorType coorType, double[]? tcpPoseInPosCoorFrame = null, double[]?
userCoorFrame = null)
```

在用户或工具坐标系中计算相对位姿/偏移。

参数	类型	说明
tcpPoseWorld	double[]	世界坐标系中的当前 TCP 位姿
offset	double[]	[dx,dy,dz,drx,dry,drz] 偏移量
coorType	RelativePoseCoorType	用户或工具坐标系
tcpPoseInPosCoorFrame	double[]?	位置坐标系中的 TCP 位姿
userCoorFrame	double[]?	用户坐标系定义

```
double[] currentPose = { 400, 0, 300, 180, 0, 0 };
double[] offset = { 50, 0, 0, 0, 0, 0 }; // X 方向偏移 +50mm

double[] newPose = await robot.CalculateRelativePoseResult(
    currentPose, offset, RelativePoseCoorType.User);

Console.WriteLine($"新位姿: [{string.Join(", ", newPose)}]");
```

运动 API 参考

本文档涵盖 CodroidCS SDK 中所有与运动相关的类型，包括关节/笛卡尔点定义、运动指令、点动参数和运动等待选项。

目录

- 1. [JointPoint](#)
- 2. [CartesianPoint](#)
- 3. [MovePoint](#)
- 4. [MoveInstruction](#)
- 5. [MoveToTarget](#)
- 6. [MoveToKind](#)
- 7. [MoveKinds](#)
- 8. [MotionWaitOptions](#)
- 9. [RobotJogParameters](#)
- 10. [RobotJogMode](#)
- 11. [RobotJogFrameType](#)

1. JointPoint

一个密封类，表示由关节角度定义的机器人目标点。

`JointPoint` 以度为单位存储 6 个关节角度，当您希望将机器人移动到精确的关节构型而无歧义时使用。

属性

属性	类型	说明
<code>JP</code>	<code>double[]</code>	6 个关节角度（单位：度）

工厂方法

方法	说明
<code>JointPoint.Degrees(double[] jointsDeg)</code>	从 6 个关节角度（度）创建。数组 必须 恰好为长度 6。

示例

```
// 创建一个包含 6 个关节角度（度）的关节点
var jp = JointPoint.Degrees(new[] { 0, 0, 90, 0, 90, 0 });

// 在运动指令中使用
await robot.Move(MoveInstruction.MovJ(jp, speed: 40, acc: 100));
```

2. CartesianPoint

一个密封类，表示由笛卡尔（工具中心点）位姿定义的机器人目标点，可选参考关节用于逆运动学求解。

`CartesianPoint` 以毫米和度为单位存储 TCP 位姿 `[x, y, z, rx, ry, rz]`。当仅提供位姿时，控制器使用默认参考关节 `[20, 20, 20, 20, 20, 20]` 进行逆运动学求解。您可以提供显式参考关节来引导 IK 求解器朝特定构型求解。

属性

属性	类型	说明
<code>Cp</code>	<code>double[]</code>	TCP 位姿 <code>[x, y, z, rx, ry, rz]</code> -- 位置单位 mm，姿态单位度
<code>Rj</code>	<code>double[]?</code>	用于逆运动学的参考关节（6 个关节角度，度）。 <code>null</code> 时使用默认值 <code>[20, 20, 20, 20, 20, 20]</code>

工厂方法

方法	说明
<code>CartesianPoint.MmDeg(double[] poseMmDeg)</code>	仅使用 TCP 位姿创建（使用默认参考关节）
<code>CartesianPoint.MmDegWithRef(double[] poseMmDeg, double[] refJointsDeg)</code>	使用 TCP 位姿和显式参考关节创建

示例

```
// 仅使用 TCP 位姿创建笛卡尔点（位置 + 姿态）
var cp = CartesianPoint.MmDeg(new[] { 400, 0, 300, 180, 0, 0 });

// 使用机器人当前状态的显式参考关节创建笛卡尔点
var refJ = robot.CriData.JointPosition;
var cpWithRef = CartesianPoint.MmDegWithRef(
    new[] { 400, 0, 300, 180, 0, 0 },
    refJ
);

// 在直线运动指令中使用
await robot.Move(MoveInstruction.MovL(cp, speed: 150, acc: 500));
```

3. MovePoint

一个密封类，内部用于运动目标点的序列化。

`MovePoint` 是向控制器发送运动指令时使用的序列化包装器。它包含可选的关节 (`Jp`)、笛卡尔 (`Cp`)、参考关节 (`Rj`) 和外部 (`Ep`) 数组。通常不需要直接创建 `MovePoint` 实例，而是使用 `MoveInstruction` 上的工厂方法。

属性

属性	类型	说明
<code>Jp</code>	<code>double[]?</code>	关节角度（度），笛卡尔目标时为 null
<code>Cp</code>	<code>double[]?</code>	TCP 位姿（mm + 度），关节目标时为 null
<code>Rj</code>	<code>double[]?</code>	用于逆运动学的参考关节
<code>Ep</code>	<code>double[]?</code>	外部轴

所有属性使用 `[JsonIgnoreWhenNull]` -- 当值为 null 时在 JSON 序列化中被忽略。

工厂方法

方法	说明
<code>MovePoint.FromJoint(JointPoint jp)</code>	从 <code>JointPoint</code> 创建
<code>MovePoint.FromCartesian(CartesianPoint cp)</code>	从 <code>CartesianPoint</code> 创建

示例

```
// 通常不会直接创建 MovePoint，而是使用 MoveInstruction 的工厂方法。

// 如果需要显式包装一个点：
var jp = JointPoint.Degrees(new[] { 0, 0, 90, 0, 90, 0 });
var movePoint = MovePoint.FromJoint(jp);

var cp = CartesianPoint.MmDeg(new[] { 400, 0, 300, 180, 0, 0 });
var movePointFromCart = MovePoint.FromCartesian(cp);
```

4. MoveInstruction

一个密封类，定义机器人运动命令中的单个运动段。

`MoveInstruction` 是构建运动路径的主要类型。每个实例描述一个运动段，包含运动类型（关节、直线或圆弧）、速度/加速度参数、混合设置以及可选的坐标系和工具偏移。

属性

属性	类型	默认值	说明
Type	string	"movJ"	运动类型： "movJ"、 "movL"、 "movC"、 "movCircle"
CircleNum	int?	null	整圆圈数（仅用于 movCircle）
Speed	double	--	速度值（直线 mm/s，关节 deg/s）
Acc	double	--	加速度值
Blend	double	--	混合半径（直线 mm，关节 deg）
RelativeBlend	double?	null	相对混合比（0--1），设置时覆盖 Blend
TargetPoint	MovePoint	--	本段的目标点
MiddlePoint	MovePoint?	null	中间/经过点（movC 和 movCircle 必需）
Coor	double[]?	null	坐标系定义
Tool	double[]?	null	工具定义

工厂方法

所有工厂方法共享可选参数： `coor`（坐标系）、 `tool`（工具偏移）和 `relativeBlend`（相对混合比）。

方法	运 动 类 型	目标类型	说明
<code>MoveInstruction.MovJ(JointPoint, speed, acc, blend, ...)</code>	关 节	JointPoint	关节运 动到关 节目标
<code>MoveInstruction.MovJ(CartesianPoint, speed, acc, blend, ...)</code>	关 节	CartesianPoint	关节运 动到笛 卡尔目 标
<code>MoveInstruction.MovL(CartesianPoint, speed, acc, blend, ...)</code>	直 线	CartesianPoint	直线运 动到笛 卡尔目 标
<code>MoveInstruction.MovL(JointPoint, speed, acc, blend, ...)</code>	直 线	JointPoint	直线运 动到关 节目标

方法	运动类型	目标类型	说明
<code>MoveInstruction.MovC(CartesianPoint middle, CartesianPoint target, speed, acc, blend, ...)</code>	圆弧	2x CartesianPoint	经过中间点到目标的圆弧运动
<code>MoveInstruction.MovCircle(CartesianPoint middle, CartesianPoint target, int circleNum, speed, acc, blend, ...)</code>	整圆	2x CartesianPoint + circleNum	整圆运动

参数参考

参数	类型	默认值	说明
<code>speed</code>	<code>double</code>	--	必需。速度（mm/s 或 deg/s）
<code>acc</code>	<code>double</code>	--	必需。加速度
<code>blend</code>	<code>double</code>	25	混合半径
<code>coor</code>	<code>double[]?</code>	<code>null</code>	坐标系
<code>tool</code>	<code>double[]?</code>	<code>null</code>	工具偏移
<code>relativeBlend</code>	<code>double?</code>	<code>null</code>	相对混合比（0--1）

示例

```
// 关节运动到关节目标（单段）
var j1 = JointPoint.Degrees(new[] { 20, 20, 90, 0, 45, 0 });
await robot.Move(MoveInstruction.MovJ(j1, speed: 40, acc: 100));

// 直线运动到笛卡尔目标（单段）
var cp = CartesianPoint.MmDeg(new[] { 400, 0, 300, 180, 0, 0 });
await robot.Move(MoveInstruction.MovL(cp, speed: 150, acc: 500));

// 多段路径：关节运动后接直线运动
var p2 = new[] { 500, 100, 400, 180, 0, 0 };
var refJ = robot.CriData.JointPosition;
await robot.Move(new[]
{
    MoveInstruction.MovJ(JointPoint.Degrees(j1), 40, 100),
    MoveInstruction.MovL(CartesianPoint.MmDegWithRef(p2, refJ), 150, 500),
});

// 圆弧运动
```

```
var mid = CartesianPoint.MmDeg(new[] { 450, 50, 350, 180, 0, 0 });
var end = CartesianPoint.MmDeg(new[] { 500, 0, 300, 180, 0, 0 });
await robot.Move(MoveInstruction.MovC(mid, end, speed: 100, acc: 300));

// 整圆运动 (2 圈)
await robot.Move(MoveInstruction.MovCircle(mid, end, circleNum: 2, speed: 80, acc: 200));

// 自定义混合和坐标系
await robot.Move(MoveInstruction.MovL(cp, speed: 100, acc: 300, blend: 10, coord:
userCoord));
```

5. MoveToTarget

一个密封类，表示预定义移动命令（回零、安全位、打包位等）的目标。

`MoveToTarget` 为 `MoveToKind` 命令包装目标点。它可以表示关节目标、笛卡尔目标或原始外部轴数据。

属性

属性	类型	说明
<code>Cp</code>	<code>double[]?</code>	笛卡尔位姿 <code>[x, y, z, rx, ry, rz]</code> (mm + 度)
<code>Jp</code>	<code>double[]?</code>	关节角度 (度)
<code>Ep</code>	<code>double[]?</code>	外部轴

工厂方法

方法	说明
<code>MoveToTarget.Joint(JointPoint jp)</code>	从 <code>JointPoint</code> 创建
<code>MoveToTarget.Cartesian(CartesianPoint cp)</code>	从 <code>CartesianPoint</code> 创建

示例

```
// 从关节点创建 moveTo 目标
var homeTarget = MoveToTarget.Joint(JointPoint.Degrees(new[] { 0, 0, 0, 0, 0, 0 }));

// 从笛卡尔点创建 moveTo 目标
var safeTarget = MoveToTarget.Cartesian(
    CartesianPoint.MmDeg(new[] { 300, 0, 400, 180, 0, 0 })
);
```

6. MoveToKind

一个枚举，指定预定义的移动目标类型。

`MoveToKind` 与机器人的 `MoveTo` 方法配合使用，命令机器人移动到已知位置或恢复程序执行。

值

名称	值	说明
<code>Stop</code>	-1	停止 moveTo 操作
<code>Home</code>	0	原点位置
<code>Safe</code>	1	安全位置
<code>Candle</code>	2	烛台（垂直）位置
<code>Pack</code>	3	打包（运输）位置
<code>JointPlanned</code>	4	关节规划运动到目标
<code>LinePlanned</code>	5	直线规划运动到目标
<code>ProgramResume</code>	6	恢复程序执行

示例

```
// 将机器人移动到原点位置
await robot.MoveTo(MoveToKind.Home);

// 停止正在进行的 moveTo 操作
await robot.MoveTo(MoveToKind.Stop);

// 使用关节规划移动到特定关节位置
var target = MoveToTarget.Joint(JointPoint.Degrees(new[] { 10, 20, 90, 0, 45, 0 }));
await robot.MoveTo(MoveToKind.JointPlanned, target);

// 使用直线规划移动到笛卡尔位置
var cartTarget = MoveToTarget.Cartesian(
    CartesianPoint.MmDeg(new[] { 400, 0, 300, 180, 0, 0 }
);
await robot.MoveTo(MoveToKind.LinePlanned, cartTarget);

// 恢复暂停的程序
await robot.MoveTo(MoveToKind.ProgramResume);
```

7. MoveKinds

一个静态类，提供运动类型标识符的字符串常量。

`MoveKinds` 定义了 `MoveInstruction.Type` 中使用的字符串常量。它们对应 CodroidCS 控制器支持的四种运动模式。

常量

名称	值	说明
<code>MovJ</code>	<code>"movJ"</code>	关节运动
<code>MovL</code>	<code>"movL"</code>	直线运动
<code>MovC</code>	<code>"movC"</code>	圆弧运动
<code>MovCircle</code>	<code>"movCircle"</code>	整圆运动

示例

```
// 比较指令的运动类型
var instruction = MoveInstruction.MovJ(
    JointPoint.Degrees(new[] { 0, 0, 90, 0, 90, 0 }),
    speed: 40, acc: 100
);

if (instruction.Type == MoveKinds.MovJ)
{
    Console.WriteLine("这是一个关节运动。");
}
else if (instruction.Type == MoveKinds.MovL)
{
    Console.WriteLine("这是一个直线运动。");
}
```

8. MotionWaitOptions

一个密封类，配置 SDK 等待运动完成的方式。

`MotionWaitOptions` 控制等待机器人运动完成时的轮询行为、容差阈值和超时时间。您可以自定义这些参数以适应不同的精度和响应要求。

属性

属性	类型	默认值	说明
<code>Timeout</code>	<code>TimeSpan</code>	60 秒	等待运动完成的最长时间
<code>PollInterval</code>	<code>TimeSpan</code>	50 ms	检查运动状态的轮询间隔

属性	类型	默认值	说明
<code>CriStaleTimeout</code>	<code>TimeSpan</code>	500 ms	CRI 数据被视为过期的最长时间
<code>SettledSamples</code>	<code>int</code>	3	确认运动完成所需的连续稳定采样数
<code>JointToleranceDeg</code>	<code>double</code>	0.2	关节位置容差（度）
<code>CartesianPositionToleranceMm</code>	<code>double</code>	1.0	笛卡尔位置容差（毫米）
<code>CartesianOrientationToleranceDeg</code>	<code>double</code>	1.0	笛卡尔姿态容差（度）

示例

```
// 使用默认等待选项（最常见）
await robot.Move(MoveInstruction.MovJ(jp, speed: 40, acc: 100));

// 为高精度运动自定义等待选项
var preciseWait = new MotionWaitOptions
{
    Timeout = TimeSpan.FromSeconds(120),
    PollInterval = TimeSpan.FromMilliseconds(20),
    SettledSamples = 5,
    JointToleranceDeg = 0.05,
    CartesianPositionToleranceMm = 0.2,
    CartesianOrientationToleranceDeg = 0.5,
};

await robot.Move(MoveInstruction.MovL(cp, speed: 50, acc: 200), preciseWait);

// 为快速运动使用较短的超时时间
var quickWait = new MotionWaitOptions
{
    Timeout = TimeSpan.FromSeconds(10),
    PollInterval = TimeSpan.FromMilliseconds(30),
};

await robot.Move(MoveInstruction.MovJ(jp, speed: 100, acc: 300), quickWait);

// 发送即忘：设置很长的超时时间以实现不等待效果
var longWait = new MotionWaitOptions
{
    Timeout = TimeSpan.FromHours(1),
};
```

9. RobotJogParameters

一个密封类，定义机器人点动（手动）运动的参数。

`RobotJogParameters` 指定手动点动操作的点动模式（关节或直线）、速度、轴索引和坐标系。

属性

属性	类型	说明
Mode	RobotJogMode	点动模式：关节或直线
Speed	double	点动速度（关节 deg/s，直线 mm/s）
Index	int	轴索引（关节模式为 0--5）
CoorType	RobotJogFrameType	坐标系类型：用户或工具
CoorId	int	坐标系 ID

工厂方法

方法	说明
RobotJogParameters.Create(RobotJogMode mode, double speed, int index, RobotJogFrameType frame, int coorId)	创建点动参数

示例

```
// 在用户坐标系 0 中以 20 deg/s 点动关节 0
var jogParams = RobotJogParameters.Create(
    mode: RobotJogMode.Joint,
    speed: 20,
    index: 0,
    frame: RobotJogFrameType.User,
    coorId: 0
);
await robot.Jog(jogParams);

// 在工具坐标系 1 中以 50 mm/s 沿 X 轴直线点动
var linearJog = RobotJogParameters.Create(
    mode: RobotJogMode.Linear,
    speed: 50,
    index: 0, // X 轴
    frame: RobotJogFrameType.Tool,
    coorId: 1
);
await robot.Jog(linearJog);
```

10. RobotJogMode

一个枚举，指定点动运动模式。

值

名称	值	说明
Joint	1	点动单个关节
Linear	2	在笛卡尔空间中直线点动

示例

```
// 在关节和直线点动模式之间切换
if (jogMode == RobotJogMode.Joint)
{
    Console.WriteLine("关节点动模式 - 移动单个关节。");
}
else if (jogMode == RobotJogMode.Linear)
{
    Console.WriteLine("直线点动模式 - 在笛卡尔空间中移动。");
}
```

11. RobotJogFrameType

一个枚举，指定点动操作的坐标系类型。

值

名称	值	说明
User	0	用户定义的坐标系
Tool	1	工具坐标系

示例

```
// 为点动操作选择坐标系
var jogParams = RobotJogParameters.Create(
    mode: RobotJogMode.Linear,
    speed: 30,
    index: 1, // Y 轴
    frame: RobotJogFrameType.User, // 使用用户坐标系
    coordId: 0
);
await robot.Jog(jogParams);
```

完整多段路径示例

以下示例演示如何使用本 API 参考中的多个类型构建完整的运动程序。

```
using CodroidCS.Sdk;

// 1. 定义路径点
var homeJoint = JointPoint.Degrees(new[] { 0, 0, 90, 0, 90, 0 });
var pickJoint = JointPoint.Degrees(new[] { 30, -20, 80, 10, 60, -5 });

var refJ = robot.CriData.JointPosition;
var pickCart = CartesianPoint.MmDegWithRef(
    new[] { 450, -100, 200, 180, 0, 0 },
    refJ
);
var placeCart = CartesianPoint.MmDeg(
    new[] { 450, 100, 200, 180, 0, 0 }
);
var viaPoint = CartesianPoint.MmDeg(
    new[] { 475, 0, 250, 180, 0, 0 }
);

// 2. 构建多段路径
var path = new[]
{
    // 关节运动回原点
    MoveInstruction.MovJ(homeJoint, speed: 60, acc: 120),

    // 关节运动到拾取位置
    MoveInstruction.MovJ(pickJoint, speed: 40, acc: 100),

    // 直线接近拾取笛卡尔点
    MoveInstruction.MovL(pickCart, speed: 80, acc: 300, blend: 5),

    // 经过路径点从拾取到放置的圆弧运动
    MoveInstruction.MovC(viaPoint, placeCart, speed: 100, acc: 300),

    // 直线运动回原点
    MoveInstruction.MovL(
        CartesianPoint.MmDeg(new[] { 300, 0, 400, 180, 0, 0 }),
        speed: 150, acc: 500
    ),
};

// 3. 使用自定义等待选项执行
var waitOptions = new MotionWaitOptions
{
    Timeout = TimeSpan.FromSeconds(90),
    SettledSamples = 4,
    JointToleranceDeg = 0.1,
};

await robot.Move(path, waitOptions);
```

```
// 4. 验证完成
```

```
Console.WriteLine("路径执行完成。");
```

数据类型与枚举

本文档提供 Codroid CRI SDK 中所有数据类型、枚举和异常的完整参考。

目录

- 1. [CommonResponse](#)
- 2. [CriRealTimeData](#)
- 3. [RobotFrame](#)
- 4. [RobotPayloadFrame](#)
- 5. [RobotParameters](#)
- 6. [RegisterReadValue](#)
- 7. [RegisterExtendArrayValueType](#)
- 8. [IoPortKind](#)
- 9. [RelativePoseCoorType](#)
- 10. [CodroidCommandException](#)
- 11. [GlobalVarSaveItem](#)
- 12. [GlobalVarRawJson](#)
- 13. [GlobalVarCatalogEntry](#)

1. CommonResponse（数据类型）

大多数 CRI SDK 方法返回的通用响应类。包含请求 ID、类型标识符、JSON 数据载荷和可选的错误消息。

属性

属性	类型	说明
id	object?	原始请求的标识符，可用于匹配请求与响应
ty	string?	标识响应的类型，用于区分不同类型的返回数据
db	JsonElement	包含实际返回数据的 JSON 元素，需根据 ty 解析
err	string?	当请求失败时包含错误描述，成功时为 null

示例：读取 db 字段

```
CommonResponse response = await client.SomeMethodAsync();

// 先检查错误
if (response.err != null)
{
    Console.WriteLine($"错误: {response.err}");
}
```

```
        return;
    }

    // 将 db 读取为特定类型
    int value = response.db.GetInt32();
    Console.WriteLine($"值: {value}");

    // 或者读取为 JSON 字符串
    string json = response.db.GetRawText();
    Console.WriteLine($"原始 JSON: {json}");
```

2. CriRealTimeData

包含来自机器人控制器的所有实时数据字段，包括关节位置、TCP 位姿、状态标志等。通过 CRI 连接持续更新。

时间戳

属性	类型	说明
TimestampMs	long	控制器端的时间戳，单位为毫秒

状态标志

属性	类型	说明
Status1Raw	ushort	控制器原始状态寄存器 1
Status2Raw	ushort	控制器原始状态寄存器 2
ProjectRunning	bool	表示当前是否有程序在运行
ProjectStopped	bool	表示程序是否已停止
ProjectPaused	bool	表示程序是否处于暂停状态
Enabling	bool	表示使能开关是否处于激活状态
NotEnabled	bool	表示机器人未处于使能状态
ManualMode	bool	表示当前是否为手动操作模式
Dragging	bool	表示机器人是否处于拖动示教状态
InMotion	bool	表示机器人当前是否有轴在运动
CollisionStopped	bool	表示机器人是否因检测到碰撞而停止
InSafetyPosition	bool	表示机器人是否已到达安全位置
HasAlarm	bool	表示控制器是否有活跃的报警信息
SimulationMode	bool	表示控制器是否运行在仿真模式下

属性	类型	说明
EmergencyStopPressed	bool	表示急停按钮是否被按下
RescueMode	bool	表示机器人是否处于碰撞救援模式
AutoMode	bool	表示控制器是否处于自动运行模式
RemoteMode	bool	表示控制器是否处于远程控制模式
RealTimeControlMode	bool	表示控制器是否处于实时控制模式
CriErrorCode	byte	CRI 协议层的错误代码

关节数据

属性	类型	说明
JointPosition	double[6]	各关节的当前角度，单位为度
JointVelocity	double[6]	各关节的当前角速度
JointOutputTorque	double[6]	各关节的当前输出力矩百分比
JointExternalForce	double[6]	各关节检测到的外部力

TCP 数据

属性	类型	说明
TcpPose	double[6]	工具中心点位姿，XYZ 单位 mm，ABC 单位度
TcpVelocity	double[6]	工具中心点的六维速度
TcpLinearVelocity	double	工具中心点的线速度标量值

外部轴

属性	类型	说明
ExternalAxisPosition	double[]	外部轴（如导轨、转台）的位置数组

方法

方法	返回类型	说明
UpdateFrom(CriRealTimeData)	void	用另一个实例的值更新当前实例的所有字段
Clone()	CriRealTimeData	创建当前实例的深拷贝副本

示例：订阅 CRI 数据并读取关节位置

```
// 订阅实时数据
client.OnCriDataReceived += (sender, data) =>
{
    long timestamp = data.TimestampMs;

    double joint1 = data.JointPosition[0];
    double joint2 = data.JointPosition[1];
    double joint3 = data.JointPosition[2];

    Console.WriteLine($"J1={joint1:F2}, J2={joint2:F2}, J3={joint3:F2}");

    double tcpX = data.TcpPose[0];
    double tcpY = data.TcpPose[1];
    double tcpZ = data.TcpPose[2];

    Console.WriteLine($"TCP X={tcpX:F2} Y={tcpY:F2} Z={tcpZ:F2}");

    if (data.HasAlarm)
    {
        Console.WriteLine("机器人有报警!");
    }
};

// 或者克隆以实现线程安全访问
CriRealTimeData snapshot = data.Clone();
```

3. RobotFrame

一个密封类，表示坐标系定义，用于工具坐标系和用户坐标系。包含 ID 和六轴位姿（位置+姿态）。

属性

属性	类型	说明
Id	int	坐标系的唯一编号
X	double	X 轴方向的偏移量（毫米）
Y	double	Y 轴方向的偏移量（毫米）
Z	double	Z 轴方向的偏移量（毫米）
A	double	绕 X 轴的旋转角度（度）
B	double	绕 Y 轴的旋转角度（度）
C	double	绕 Z 轴的旋转角度（度）

示例

```
RobotFrame tool = robotParams.Tool[0];
Console.WriteLine($"Tool {tool.Id}: X={tool.X}, Y={tool.Y}, Z={tool.Z}");
Console.WriteLine($"  A={tool.A}, B={tool.B}, C={tool.C}");
```

4. RobotPayloadFrame

一个密封类，表示负载定义，包括质量和质心坐标。

属性

属性	类型	说明
Id	int	负载配置的唯一编号
M	double	负载的质量（千克）
Mx	double	质心在 X 方向的偏移（毫米）
My	double	质心在 Y 方向的偏移（毫米）
Mz	double	质心在 Z 方向的偏移（毫米）

示例

```
RobotPayloadFrame payload = robotParams.Payload[0];
Console.WriteLine($"Payload {payload.Id}: Mass={payload.M}kg");
Console.WriteLine($"  CoM: ({payload.Mx}, {payload.My}, {payload.Mz})");
```

5. RobotParameters

一个密封类，包含机器人的完整参数集，包括工具、负载和坐标系的默认 ID，以及所有已配置的坐标系列表。

属性

属性	类型	说明
DefaultToolId	int	当前激活的工具坐标系编号
DefaultPayloadId	int	当前激活的负载配置编号
DefaultCoordinateId	int	当前激活的用户坐标系编号
MaxPayload	double	机器人允许的最大负载质量（千克）
Tool	List<RobotFrame>	所有已配置的工具坐标系
Payload	List<RobotPayloadFrame>	所有已配置的负载参数

属性	类型	说明
Coordinate	List<RobotFrame>	所有已配置的用户坐标系

示例：读取机器人参数

```
RobotParameters parameters = await client.GetRobotParametersAsync();

Console.WriteLine($"默认工具 ID: {parameters.DefaultToolId}");
Console.WriteLine($"默认负载 ID: {parameters.DefaultPayloadId}");
Console.WriteLine($"默认坐标系 ID: {parameters.DefaultCoordinateId}");
Console.WriteLine($"最大负载: {parameters.MaxPayload} kg");

foreach (RobotFrame tool in parameters.Tool)
{
    Console.WriteLine($" [{tool.Id}] X={tool.X}, Y={tool.Y}, Z={tool.Z}, "
        + $"A={tool.A}, B={tool.B}, C={tool.C}");
}

foreach (RobotPayloadFrame payload in parameters.Payload)
{
    Console.WriteLine($" [{payload.Id}] M={payload.M}kg, "
        + $"CoM=({payload.Mx}, {payload.My}, {payload.Mz})");
}

foreach (RobotFrame coord in parameters.Coordinate)
{
    Console.WriteLine($" [{coord.Id}] X={coord.X}, Y={coord.Y}, Z={coord.Z}, "
        + $"A={coord.A}, B={coord.B}, C={coord.C}");
}
```

6. RegisterReadValue

一个只读结构体，表示从控制器寄存器读取的值，提供将值转换为常见类型的辅助方法。

属性

属性	类型	说明
Address	int	读取的寄存器地址编号
Value	JsonElement	寄存器的原始值，以 JSON 元素形式存储

方法

方法	返回类型	说明
GetInt32()	int	直接将寄存器值转换为 32 位整数，转换失败时抛出异常
GetDouble()	double	直接将寄存器值转换为双精度浮点数

方法	返回类型	说明
<code>TryGetInt32(out int)</code>	<code>bool</code>	安全尝试转换，失败时返回 <code>false</code> 而不抛出异常

示例

```
List<RegisterReadValue> values = await client.ReadRegistersAsync(address: 0, count: 5);

foreach (RegisterReadValue reg in values)
{
    Console.WriteLine($"寄存器 [{reg.Address}] 原始值: {reg.Value}");

    int intVal = reg.GetInt32();
    Console.WriteLine($"  作为 Int32: {intVal}");

    if (reg.TryGetInt32(out int safeVal))
        Console.WriteLine($"  安全 Int32: {safeVal}");
    else
        Console.WriteLine($"  无法转换为 Int32");

    double dblVal = reg.GetDouble();
    Console.WriteLine($"  作为 Double: {dblVal}");
}
```

7. RegisterExtendArrayValueType

一个静态类，定义扩展寄存器数组中使用的数据类型常量。

常量

常量	值	说明
<code>Bool</code>	<code>0</code>	布尔值， <code>true</code> 或 <code>false</code>
<code>UInt8</code>	<code>1</code>	无符号 8 位整数，范围 0 ~ 255
<code>Int8</code>	<code>2</code>	有符号 8 位整数，范围 -128 ~ 127
<code>UInt16</code>	<code>3</code>	无符号 16 位整数，范围 0 ~ 65535
<code>Int16</code>	<code>4</code>	有符号 16 位整数，范围 -32768 ~ 32767
<code>UInt32</code>	<code>5</code>	无符号 32 位整数，范围 0 ~ 4294967295
<code>Int32</code>	<code>6</code>	有符号 32 位整数，范围 -2147483648 ~ 2147483647
<code>Float32</code>	<code>7</code>	32 位浮点数，单精度浮点数

示例

```
await client.WriteExtendRegisterAsync(  
    address: 0,  
    values: new[] { 3.14f },  
    valueType: RegisterExtendArrayValueType.Float32  
);
```

8. IoPortKind

一个静态类，定义控制器上可用的不同 I/O 端口类型常量。

常量

常量	值	说明
Di	"DI"	数字输入端口，用于读取开关量信号
Do	"DO"	数字输出端口，用于控制开关量信号
Ai	"AI"	模拟输入端口，用于读取连续量信号
Ao	"AO"	模拟输出端口，用于输出连续量信号

9. RelativePoseCoorType

一个枚举，指定相对位姿所表达的坐标系。

值

名称	值	说明
User	0	相对位姿在用户坐标系下表达
Tool	1	相对位姿在当前工具坐标系下表达

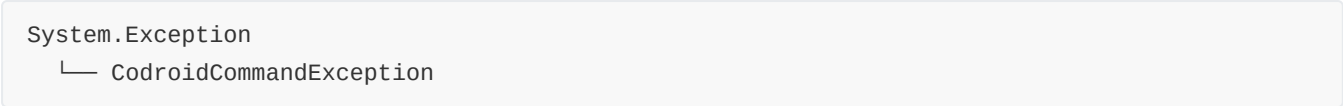
示例

```
// 在工具坐标系下做相对运动  
await client.MoveRelativeAsync(  
    pose: new[] { 0, 0, 10, 0, 0, 0 },  
    coorType: RelativePoseCoorType.Tool  
);  
  
// 在用户坐标系下做相对运动  
await client.MoveRelativeAsync(  
    pose: new[] { 10, 0, 0, 0, 0, 0 },  
    coorType: RelativePoseCoorType.User  
);
```

10. CodroidCommandException

当 CRI 命令失败时抛出的密封异常类。提供关于失败的详细上下文，包括请求 ID、命令类型、控制器错误消息和完整响应。

继承



属性

属性	类型	说明
RequestId	int	用于匹配请求与响应的标识符
CommandType	string	字符串标识符，表示哪个 CRI 命令失败
ControllerError	string?	控制器返回的错误描述，可能为 null
Response	CommonResponse?	包含原始响应数据，可用于进一步诊断

构造函数

```
public CodroidCommandException(
    int requestId,
    string commandType,
    string? controllerError,
    CommonResponse? response
)
```

示例：捕获和检查异常

```
try
{
    await client.MoveJointAsync(target: new[] { 0, 0, 90, 0, 90, 0 });
}
catch (CodroidCommandException ex)
{
    Console.WriteLine("命令失败!");
    Console.WriteLine($"  请求 ID: {ex.RequestId}");
    Console.WriteLine($"  命令类型: {ex.CommandType}");
    Console.WriteLine($"  控制器错误: {ex.ControllerError}");

    if (ex.Response != null)
    {
        Console.WriteLine($"  响应错误: {ex.Response.err}");
        Console.WriteLine($"  响应数据: {ex.Response.db.GetRawText()}");
    }
}
```

```
        throw;
    }
}
```

11. GlobalVarSaveItem

一个只读记录结构体，用于指定要保存或写入控制器的全局变量，包括其名称、值和可选备注。

构造函数

```
public GlobalVarSaveItem(string Name, object Value, string? Remark = null)
```

参数	类型	必需	说明
Name	string	是	全局变量的标识名称
Value	object	是	变量的值，支持多种类型
Remark	string?	否	变量的描述或注释信息

示例

```
var items = new List<GlobalVarSaveItem>
{
    new GlobalVarSaveItem("Counter", 42, "生产计数"),
    new GlobalVarSaveItem("Speed", 50.5, "运动速度"),
    new GlobalVarSaveItem("Flag", true)
};

await client.SaveGlobalVarsAsync(items);
```

12. GlobalVarRawJson

一个只读记录结构体，封装原始 JSON 字符串字面量，用于写入具有复杂或自定义 JSON 结构的全局变量。

构造函数

```
public GlobalVarRawJson(string Literal)
```

参数	类型	必需	说明
Literal	string	是	直接传递给控制器的 JSON 字面量

示例

```
var rawJson = new GlobalVarRawJson(
    """
    {"positions": [1.0, 2.0, 3.0], "enabled": true}
    """
);

await client.WriteGlobalVarRawAsync("MyConfig", rawJson);
```

13. GlobalVarCatalogEntry

一个密封类，表示全局变量目录中的单个条目，包含变量的当前值和可选备注。

属性

属性	类型	默认值	说明
Value	JsonElement	--	变量当前存储的值，以JSON 元素形式表示
Remark	string	""	变量的描述信息，默认为空字符串

示例

```
Dictionary<string, GlobalVarCatalogEntry> catalog =
    await client.GetGlobalVarCatalogAsync();

foreach (var (name, entry) in catalog)
{
    Console.WriteLine($"变量: {name}");
    Console.WriteLine($"  值: {entry.Value.GetRawText()}");
    Console.WriteLine($"  备注: {entry.Remark}");
}
```

CRI 实时数据与控制 API 参考

本文档涵盖 CRI（Codroid 实时接口）API，用于实时机器人控制、轨迹生成与数据解析。

目录

- 1. [CriRealtimeDispatcher](#)
- 2. [TrajectoryGenerator](#)
- 3. [TrajectoryRequest](#)
- 4. [TrajectoryPoint](#)
- 5. [TrajectorySpace](#)
- 6. [TrajectoryProfile](#)
- 7. [CriRealtimePacketParser](#)
- 8. [完整 CRI 控制流程示例](#)

1. CriRealtimeDispatcher

密封类，实现 IDisposable。

基于 UDP 的命令调度器，向机器人控制器发送实时运动指令。支持单帧命令和完整轨迹回放，可配置 SI 单位转换。

常量

常量	类型	值	说明
DefaultControllerUdpPort	int	9030	CRI 命令默认 UDP 端口
CommandPacketLength	int	64	每个命令包的固定长度

构造函数

```
CriRealtimeDispatcher(string controllerIp, int controllerUdpPort = 9030, bool convertToSi = true)
```

参数	类型	默认值	说明
controllerIp	string	(必需)	机器人控制器的 IP 地址
controllerUdpPort	int	9030	发送命令的 UDP 端口
convertToSi	bool	true	若为 true，发送前将度转换为弧度、毫米转换为米

方法

SendCommand

```
Task SendCommand(  
    IReadOnlyList<double> position6,  
    TrajectorySpace space,  
    CancellationToken ct = default  
)
```

向控制器发送单帧位置命令。 `position6` 列表必须包含恰好 6 个元素。

参数	类型	说明
<code>position6</code>	<code>IReadOnlyList<double></code>	目标位置，必须为 6 个元素
<code>space</code>	<code>TrajectorySpace</code>	坐标空间： <code>Joint</code> 或 <code>Cartesian</code>
<code>ct</code>	<code>CancellationToken</code>	取消令牌

```
var dispatcher = new CriRealtimeDispatcher("192.168.8.136");  
await dispatcher.SendCommand(  
    new double[] { 0, 0, 90, 0, 90, 0 },  
    TrajectorySpace.Joint  
);
```

SendTrajectory

```
Task SendTrajectory(  
    IEnumerable<TrajectoryPoint> trajectory,  
    TrajectorySpace space,  
    int periodMs,  
    CancellationToken ct = default  
)
```

以固定时间间隔向控制器发送完整轨迹。

参数	类型	说明
<code>trajectory</code>	<code>IEnumerable<TrajectoryPoint></code>	要发送的轨迹点序列
<code>space</code>	<code>TrajectorySpace</code>	坐标空间： <code>Joint</code> 或 <code>Cartesian</code>
<code>periodMs</code>	<code>int</code>	相邻点之间的时间间隔（毫秒）
<code>ct</code>	<code>CancellationToken</code>	取消令牌

```
using var dispatcher = new CriRealtimeDispatcher("192.168.8.136");  
await dispatcher.SendTrajectory(trajectory, TrajectorySpace.Joint, periodMs: 4);
```

Dispose

```
void Dispose()
```

关闭底层 UDP 套接字并释放资源。

2. TrajectoryGenerator

静态类。使用可配置的运动曲线（三次或梯形）在两个位置之间生成平滑轨迹。

Generate

```
static IEnumerable<TrajectoryPoint> Generate(  
    IReadOnlyList<double> start,  
    IReadOnlyList<double> target,  
    TrajectoryRequest request  
)
```

根据 request 中的参数，生成从 start 到 target 的轨迹。

参数	类型	说明
start	IReadOnlyList<double>	起始位置
target	IReadOnlyList<double>	目标位置
request	TrajectoryRequest	轨迹生成参数

返回值: IReadOnlyList<TrajectoryPoint> -- 轨迹点序列

```
var start = new double[] { 0, 0, 90, 0, 90, 0 };  
var target = new double[] { 10, 20, 45, 0, 60, 30 };  
  
var request = new TrajectoryRequest  
{  
    Space = TrajectorySpace.Joint,  
    Profile = TrajectoryProfile.Cubic,  
    FrequencyHz = 250,  
    Speed = 30  
};  
  
var trajectory = TrajectoryGenerator.Generate(start, target, request).ToList();  
Console.WriteLine($"生成了 {trajectory.Count} 个轨迹点");
```

3. TrajectoryRequest

密封类。控制轨迹生成的参数。

属性

属性	类型	默认值	说明
Space	TrajectorySpace	(无)	轨迹的坐标空间
FrequencyHz	double	250.0	采样频率（赫兹）
Speed	double?	null	目标速度（与 DurationSeconds 互斥）
DurationSeconds	double?	null	总时长（秒，与 Speed 互斥）
Profile	TrajectoryProfile	Cubic	运动曲线类型
Acceleration	double	1000.0	梯形曲线的加速度值

注意: Speed 和 DurationSeconds 互斥，只能设置其中一个。

```
var requestBySpeed = new TrajectoryRequest
{
    Space = TrajectorySpace.Joint,
    FrequencyHz = 250,
    Speed = 30,
    Profile = TrajectoryProfile.Trapezoidal,
    Acceleration = 800
};

var requestByDuration = new TrajectoryRequest
{
    Space = TrajectorySpace.Joint,
    FrequencyHz = 250,
    DurationSeconds = 2.5,
    Profile = TrajectoryProfile.Cubic
};
```

4. TrajectoryPoint

密封类。表示轨迹中的单个点，包含时间戳和位置数组。

属性

属性	类型	默认值	说明
TimeSeconds	double	0.0	从轨迹开始的时间偏移（秒）
Position	double[]	[]	位置值（关节角度或笛卡尔坐标）

```
var point = new TrajectoryPoint
{
    TimeSeconds = 0.016,
    Position = new double[] { 0.5, 1.0, 45.0, 0.0, 30.0, 10.0 }
};
```

5. TrajectorySpace

枚举。定义轨迹位置使用的坐标空间。

名称	值	说明
Joint	0	关节空间：位置为关节角度
Cartesian	1	笛卡尔空间：位置为工具位姿 (X, Y, Z, Rx, Ry, Rz)

6. TrajectoryProfile

枚举。定义轨迹生成中使用的运动曲线形状。

名称	值	说明
Cubic	0	三次多项式曲线：平滑加减速
Trapezoidal	1	梯形速度曲线：恒加速段、匀速段、恒减速段

7. CriRealtimePacketParser

静态类。将从控制器接收的原始 CRI 数据包解析为结构化的 `CriRealTimeData` 对象。自动转换单位（米转毫米，弧度转度）。

常量

常量	类型	值	说明
PacketLength	int	308	CRI 数据包的预期字节长度
DefaultDecimalPlaces	int	3	默认四舍五入小数位数

Parse

```
static CriRealTimeData Parse(byte[] packet)
```

参数	类型	说明
packet	byte[]	原始 CRI 数据包（必须为 308 字节）

返回值: `CriRealTimeData` -- 解析后的实时数据对象

```
byte[] rawPacket = ReceiveCriPacket();
var data = CriRealtimePacketParser.Parse(rawPacket);

Console.WriteLine($"关节位置: [{string.Join(", ", data.JointPosition)}]");
Console.WriteLine($"TCP 位姿: [{string.Join(", ", data.TcpPose)}]");
```

8. 完整 CRI 控制流程示例

```
using Codroid.CRI;
using Codroid.Robot;

var robot = new CodroidRobot("192.168.8.150");

// 步骤 1: 启动 CRI 数据推送
await robot.StartCriDataPush("192.168.8.150", 18888);

// 步骤 2: 读取当前位置
double[] start = robot.CriData.JointPosition;
Console.WriteLine($"当前位置: [{string.Join(", ", start)}]");

double[] target = new[] { 0, 0, 90, 0, 90, 0 };

// 步骤 3: 生成轨迹
var request = new TrajectoryRequest
{
    Space = TrajectorySpace.Joint,
    Profile = TrajectoryProfile.Cubic,
    FrequencyHz = 250,
    Speed = 30
};

var trajectory = TrajectoryGenerator.Generate(start, target, request).ToList();
Console.WriteLine($"生成了 {trajectory.Count} 个点");

// 步骤 4: 启动 CRI 控制
await robot.StartCriControl(filterType: 1, durationMs: 4, startBuffer: 5);

try
{
    // 步骤 5: 下发轨迹
    using var dispatcher = new CriRealtimeDispatcher("192.168.8.136");
    await dispatcher.SendTrajectory(trajectory, TrajectorySpace.Joint, periodMs: 4);

    Console.WriteLine("轨迹执行完成");
}
finally
{
    // 步骤 6: 停止 CRI 控制
    await robot.StopCriControl();
}
```

```
await robot.StopCriDataPush("192.168.8.150", 18888);

Console.WriteLine("CRI 控制已停止");
}
```

工作流程图



重要: 始终在 `finally` 块中停止 CRI 控制和数据推送，确保即使轨迹执行过程中发生异常也能安全关闭。

IO 与寄存器 API 参考

IO 操作

所有 IO 方法均位于 `CodroidClient` 上。

GetDi -- 读取数字输入

读取数字输入端口。返回 `0` 或 `1`。

```
int di0 = await robot.GetDi(0);
Console.WriteLine($"DI 0 = {di0}");
```

```
Task<int> GetDi(int port)
```

GetDo -- 读取数字输出

读取数字输出端口的当前状态。返回 `0` 或 `1`。

```
int do10 = await robot.GetDo(10);
Console.WriteLine($"DO 10 = {do10}");
```

```
Task<int> GetDo(int port)
```

GetAi -- 读取模拟输入

读取模拟输入端口。返回浮点值。

```
double ai1 = await robot.GetAi(1);
Console.WriteLine($"AI 1 = {ai1:F3}");
```

```
Task<double> GetAi(int port)
```

GetAo -- 读取模拟输出

读取模拟输出端口的当前值。

```
double ao2 = await robot.GetAo(2);
Console.WriteLine($"AO 2 = {ao2:F3}");
```

```
Task<double> GetAo(int port)
```

SetDo -- 写入数字输出

写入数字输出。`value` 必须为 `0` 或 `1`。

```
await robot.SetDo(10, 1); // 设为 ON
await robot.SetDo(10, 0); // 设为 OFF
```

```
Task<CommonResponse> SetDo(int port, int value)
```

如果 `value` 不是 `0` 或 `1`，抛出 `ArgumentOutOfRangeException`。

SetAo -- 写入模拟输出

写入模拟输出值。

```
await robot.SetAo(2, 3.14);
```

```
Task<CommonResponse> SetAo(int port, double value)
```

GetIoValues -- 批量读取 IO

在一次请求中批量读取多个 IO 点。返回原始 `CommonResponse`，其 `db` 为 JSON 数组。

```
var resp = await robot.GetIoValues(new List<(string Type, int Port)>
{
    (IoPortKind.Di, 0),
    (IoPortKind.Do, 10),
    (IoPortKind.Ai, 1),
    (IoPortKind.Ao, 2),
});
Console.WriteLine(resp.db.GetRawText());

int di0 = IoGetResponseParser.ParseDigital(resp, IoPortKind.Di, 0);
double ao2 = IoGetResponseParser.ParseAnalog(resp, IoPortKind.Ao, 2);
```

```
Task<CommonResponse> GetIoValues(IReadOnlyList<(string Type, int Port)> pins)
```

IO 辅助类型

IoPortKind

IOManager 协议中 `type` 字段的常量。

常量	值	说明
<code>IoPortKind.Di</code>	<code>"DI"</code>	数字输入

常量	值	说明
<code>IoPortKind.Do</code>	<code>"DO"</code>	数字输出
<code>IoPortKind.Ai</code>	<code>"AI"</code>	模拟输入
<code>IoPortKind.Ao</code>	<code>"AO"</code>	模拟输出

IoGetResponseParser

解析 `GetIoValues` 响应中的 `db` 字段。

```
int di = IoGetResponseParser.ParseDigital(response, IoPortKind.Di, 0);
double ai = IoGetResponseParser.ParseAnalog(response, IoPortKind.Ai, 1);

var pins = new List<(string, int)> { (IoPortKind.Di, 0), (IoPortKind.Do, 10) };
JsonElement query = IoGetResponseParser.BuildGetQuery(pins);
```

方法	返回值	说明
<code>ParseDigital(response, ioType, port)</code>	<code>int</code>	提取 DI/DO 的 <code>0</code> 或 <code>1</code>
<code>ParseAnalog(response, ioType, port)</code>	<code>double</code>	提取 AI/AO 的浮点值
<code>BuildGetQuery(pins)</code>	<code>JsonElement</code>	构建批量 IO 查询的 JSON 数组

寄存器操作

所有寄存器方法均位于 `CodroidClient` 上。

GetRegisterValue -- 读取单个寄存器

```
RegisterReadValue reg = await robot.GetRegisterValue(49100);

if (reg.TryGetInt32(out int intVal))
    Console.WriteLine($"地址 {reg.Address} = {intVal} (整数)");
else
    Console.WriteLine($"地址 {reg.Address} = {reg.GetDouble()} (浮点)");
```

```
Task<RegisterReadValue> GetRegisterValue(int address)
```

GetRegisterValues -- 批量读取寄存器

```
var regs = await robot.GetRegisterValues(new[] { 49100, 49102, 49104 });

foreach (var r in regs)
{
    if (r.TryGetInt32(out int v))
        Console.WriteLine($" 地址 {r.Address}: {v}");
    else
        Console.WriteLine($" 地址 {r.Address}: {r.GetDouble():G}");
}
```

```
Task<IReadOnlyList<RegisterReadValue>> GetRegisterValues(IReadOnlyList<int> addresses)
```

SetRegisterValue (int) -- 写入寄存器整型值

```
await robot.SetRegisterValue(49100, 520);
await robot.SetRegisterValue(49100, 0); // 清零
```

```
Task<CommonResponse> SetRegisterValue(int address, int value)
```

SetRegisterValue (double) -- 写入寄存器浮点值

```
await robot.SetRegisterValue(49300, 520.52);
await robot.SetRegisterValue(49300, 0.0); // 清零
```

```
Task<CommonResponse> SetRegisterValue(int address, double value)
```

SetExtendArrayType -- 设置扩展数组元素类型

设置扩展数组元素的数据类型。索引范围：0~999。

```
await robot.SetExtendArrayType(0, RegisterExtendArrayType.Int32);
await robot.SetExtendArrayType(5, RegisterExtendArrayType.Float32);
```

```
Task<CommonResponse> SetExtendArrayType(int index, string type)
```

如果 `index` 不在 0~999 范围内，抛出 `ArgumentOutOfRangeException`。

RemoveExtendArray -- 删除扩展数组元素

删除扩展数组元素并重置其数据。索引范围：0~999。

```
await robot.RemoveExtendArray(0);
```

```
Task<CommonResponse> RemoveExtendArray(int index)
```

RegisterReadValue 结构体

属性	类型	说明
Address	int	寄存器地址
Value	JsonElement	控制器返回的原始 JSON 值

方法	返回值	说明
GetInt32()	int	读为整数；不可转换时抛出异常
GetDouble()	double	读为浮点数
TryGetInt32(out int value)	bool	尝试读为整数，不抛异常

RegisterExtendArrayValueType 常量

常量	协议值
RegisterExtendArrayValueType.Bool	"Bool"
RegisterExtendArrayValueType.UInt8	"UInt8"
RegisterExtendArrayValueType.Int8	"Int8"
RegisterExtendArrayValueType.UInt16	"UInt16"
RegisterExtendArrayValueType.Int16	"Int16"
RegisterExtendArrayValueType.UInt32	"UInt32"
RegisterExtendArrayValueType.Int32	"Int32"
RegisterExtendArrayValueType.Float32	"Float32"

完整示例：IO 与寄存器读写

```
using Codroid;

ConsoleUtf8.InitConsoleUtf8();

var robot = new CodroidClient("192.168.8.136");

try
{
    await robot.ConnectRemoteAndSwitchOn();

    // --- IO ---
    int di0 = await robot.GetDi(0);
    Console.WriteLine($"DI 0 = {di0}");
    await robot.SetDo(10, di0);

    double ai1 = await robot.GetAi(1);
    double ao2 = await robot.GetAo(2);
    Console.WriteLine($"AI 1 = {ai1:F3}, AO 2 = {ao2:F3}");

    var batch = await robot.GetIoValues(new List<(string, int)>
    {
        (IoPortKind.Di, 0),
        (IoPortKind.Do, 10),
    });
    Console.WriteLine("批量结果: " + batch.db.GetRawText());

    // --- 寄存器 ---
    RegisterReadValue r = await robot.GetRegisterValue(49100);
    Console.WriteLine($"寄存器 49100 = {r.GetDouble():G}");

    var regs = await robot.GetRegisterValues(new[] { 49100, 49102, 49104 });
    foreach (var rv in regs)
        Console.WriteLine($" {rv.Address}: {rv.GetDouble():G}");

    await robot.SetRegisterValue(49100, 520);
    await robot.SetRegisterValue(49300, 520.52);

    await robot.SetExtendArrayType(0, RegisterExtendArrayValueType.Int32);
    await robot.RemoveExtendArray(0);
}
finally
{
    robot.Disconnect();
}
```

辅助工具 API 参考

发布/订阅（TCP 主题推送）

控制器通过 TCP 推送状态变更通知。使用 `SubscribePublishTopic` 注册特定主题的回调。

SubscribePublishTopic -- 订阅主题

```
Task<PublishTopicSubscription> SubscribePublishTopic(  
    string topicTy,  
    Action<PublishNotification> handler,  
    int tcMilliseconds = 100)
```

订阅 TCP 主题推送。首次在连接上调用时发送订阅帧（无 `id`）。之后匹配 `topicTy` 的推送将在线程池上分发给 `handler`。

参数	说明
<code>topicTy</code>	主题名，如 <code>PublishTopics.RobotStatus</code>
<code>handler</code>	处理通知的回调；不应长时间阻塞
<code>tcMilliseconds</code>	协议 <code>tc</code> 字段，毫秒；默认 100

返回 `PublishTopicSubscription`（可释放）。调用 `Dispose()` 取消本地回调注册。

示例：订阅 RobotStatus

```
using Codroid;  
  
var robot = new CodroidClient("192.168.8.136");  
await robot.Connect();  
  
int count = 0;  
using var sub = await robot.SubscribePublishTopic(  
    PublishTopics.RobotStatus,  
    msg =>  
    {  
        int n = Interlocked.Increment(ref count);  
        Console.WriteLine($"[{n}] ty={msg.Ty}");  
        if (msg.Db.ValueKind != JsonValueKind.Undefined)  
            Console.WriteLine("  db: " + msg.Db.GetRawText());  
    });  
  
await Task.Delay(TimeSpan.FromSeconds(10));  
Console.WriteLine($"收到 {Volatile.Read(ref count)} 次推送。");  
sub.Dispose();  
robot.Disconnect();
```

PublishTopicSubscription

成员	说明
TopicTy	主题名（string）
Dispose()	取消本地回调注册

PublishNotification

属性	类型	说明
Ty	string	主题类型，如 "publish/RobotStatus"
Db	JsonElement	业务载荷；缺省时为 Undefined
RawJson	string	本条消息的完整 JSON 文本

PublishTopics 常量

常量	值	说明
PublishTopics.ProjectState	"publish/ProjectState"	工程运行状态
PublishTopics.VarUpdate	"publish/VarUpdate"	全局变量变更
PublishTopics.RobotStatus	"publish/RobotStatus"	机器人状态
PublishTopics.RobotPosture	"publish/RobotPosture"	机器人姿态
PublishTopics.RobotCoordinate	"publish/RobotCoordinate"	坐标数据
PublishTopics.Log	"publish/Log"	日志消息
PublishTopics.Error	"publish/Error"	错误通知

全局变量

GetGlobalVars -- 读取全局变量

```
CommonResponse resp = await robot.GetGlobalVars();
Console.WriteLine(resp.db.GetRawText());
```

```
Task<CommonResponse> GetGlobalVars()
```

GetGlobalVarsCatalog -- 读取全局变量目录

```
var catalog = await robot.GetGlobalVarsCatalog();

foreach (var kv in catalog)
{
    Console.WriteLine($"  名称: {kv.Key}");
    Console.WriteLine($"  值: {kv.Value.Value.GetRawText()}");
    Console.WriteLine($"  备注: {kv.Value.Remark}");
}
```

```
Task<IReadOnlyDictionary<string, GlobalVarCatalogEntry>> GetGlobalVarsCatalog()
```

SaveGlobalVar -- 保存单个全局变量

```
await robot.SaveGlobalVar("my_counter", 100, "test remark");
await robot.SaveGlobalVar("my_name", "hello_codroid");
await robot.SaveGlobalVar("my_arr", new[] { 1, 2, 3 });
await robot.SaveGlobalVar("my_map", new Dictionary<string, int> { ["x"] = 10 });
await robot.SaveGlobalVar("my_pose",
    new GlobalVarRawJson("{\"jp\": [1, 2, 3, 4, 5, 6]}"));
```

```
Task<CommonResponse> SaveGlobalVar(string name, object value, string? remark = null)
```

变量名由 `GlobalVarNaming.Validate()` 校验：必须以字母或下划线开头，仅含 `[A-Za-z0-9_]`，不得以 `_` 开头，且不得与保留标识符冲突。

SaveGlobalVars -- 批量保存全局变量

```
await robot.SaveGlobalVars(new[]
{
    new GlobalVarSaveItem("sdk_test_int", 100, "整数测试"),
    new GlobalVarSaveItem("sdk_test_float", 90.4, "浮点测试"),
    new GlobalVarSaveItem("sdk_test_str", "hello", "字符串测试"),
    new GlobalVarSaveItem("sdk_test_arr", new[] { 1, 2, 3 }),
});
```

```
Task<CommonResponse> SaveGlobalVars(IReadOnlyCollection<GlobalVarSaveItem> items)
```

RemoveGlobalVars -- 删除全局变量

```
await robot.RemoveGlobalVars(new[] { "sdk_test_int", "sdk_test_float" });
```

```
Task<CommonResponse> RemoveGlobalVars(IEnumerable<string> names)
```

删除不存在的名称不会报错。

全局变量辅助类型

GlobalVarSaveItem

```
public readonly record struct GlobalVarSaveItem(  
    string Name,  
    object Value,  
    string? Remark = null);
```

字段	说明
Name	变量名（会校验）
Value	任意可 JSON 序列化对象，或 GlobalVarRawJson
Remark	可选备注（可中文）；null/空白则不发送 nm 字段

GlobalVarRawJson

```
var raw = new GlobalVarRawJson("{\"jp\": [1,2,3,4,5,6]}");  
await robot.SaveGlobalVar("my_pose", raw);
```

GlobalVarCatalogEntry

属性	类型	说明
Value	JsonElement	变量的值
Remark	string	备注字符串；无则为空

GlobalVarNaming

```
GlobalVarNaming.Validate("my_var");           // OK  
GlobalVarNaming.Validate("__bad");             // 抛出：双下划线  
GlobalVarNaming.Validate("if");               // 抛出：保留字  
  
ICollection<string> reserved = GlobalVarNaming.ReservedNames;
```

成员	说明
Validate(string name)	无效名称时抛出 ArgumentException
ReservedNames	保留标识符的只读集合

运动学

AposToCpos -- 正运动学

```
var jp = new[] { 0.0, 0.0, 90.0, 0.0, 90.0, 0.0 };
var coor = new[] { 0.0, 0.0, 0.0, 0.0, 0.0, 0.0 };
var tool = new[] { 0.0, 0.0, 0.0, 0.0, 0.0, 0.0 };

double[] pose = await robot.AposToCposPose(jp, coor, tool);
Console.WriteLine($"TCP: [{string.Join(", ", pose)}]");

CommonResponse resp = await robot.AposToCpos(jp, coor, tool);
```

```
Task<double[]> AposToCposPose(
    double[] jointDegrees,
    double[] userFrame,
    double[] toolFrame,
    double[]? externalAxisPositions = null)
```

所有向量必须恰好 6 个元素。单位：关节为度，坐标系为 mm + deg。

CposToApos -- 逆运动学

```
var cp = new[] { 927.503, 214.5, 898.998, 179.999, 0.0, -90.0 };
var rj = new[] { 10.0, 20.0, 30.0, 40.0, 50.0, 60.0 };

try
{
    double[] joints = await robot.CposToAposJoints(cp, rj);
    Console.WriteLine($"关节角 (度): [{string.Join(", ", joints)}]");
}
catch (InvalidOperationException)
{
    Console.WriteLine("未找到解。请尝试不同的参考关节。");
}
```

```
Task<double[]> CposToAposJoints(
    double[] cartesianMmDeg,
    double[] referenceJointDegrees,
    double[]? externalAxisPositions = null)
```

`referenceJointDegrees` 用作起始猜测。如果控制器返回空数组，将抛出 `InvalidOperationException`。

CalculateRelativePose / CalculateRelativePoseResult

```
var currentPose = new[] { 927.503, 214.5, 898.998, 179.999, 0.0, -90.0 };
var offset = new[] { 0.0, 0.0, -300.0, 0.0, 0.0, 0.0 };

double[] result = await robot.CalculateRelativePoseResult(
    currentPose, offset, RelativePoseCoorType.User);
Console.WriteLine($"结果: [{string.Join(", ", result)}]");

double[] toolResult = await robot.CalculateRelativePoseResult(
    currentPose, offset, RelativePoseCoorType.Tool);
```

```
Task<double[]> CalculateRelativePoseResult(
    double[] tcpPoseWorld,
    double[] offset,
    RelativePoseCoorType coorType,
    double[]? tcpPoseInPosCoorFrame = null,
    double[]? userCoorFrame = null)
```

ConsoleUtf8

将 `Console.InputEncoding` 和 `Console.OutputEncoding` 设为 UTF-8。在 Windows 上防止中文乱码。在 Linux/macOS 上为空操作。

```
public static void InitConsoleUtf8()
```

```
using Codroid;

// 在程序入口调用
ConsoleUtf8.InitConsoleUtf8();

var robot = new CodroidClient("192.168.8.136");
```

完整示例：辅助工具 API

```
using System;
using System.Text.Json;
using System.Threading;
using Codroid;

ConsoleUtf8.InitConsoleUtf8();

var robot = new CodroidClient("192.168.8.136");

try
{
    await robot.ConnectRemoteAndSwitchOn();
```

```

// --- 发布/订阅 ---
using var sub = await robot.SubscribePublishTopic(
    PublishTopics.RobotStatus,
    msg => Console.WriteLine($"推送: ty={msg.Ty}"));

// --- 全局变量 ---
await robot.SaveGlobalVar("sdk_demo", 42, "demo variable");
var catalog = await robot.GetGlobalVarsCatalog();
if (catalog.TryGetValue("sdk_demo", out var entry))
    Console.WriteLine($"sdk_demo = {entry.Value.GetRawText()}");

await robot.RemoveGlobalVars(new[] { "sdk_demo" });

// --- 运动学 ---
var jp = new[] { 0.0, 0.0, 90.0, 0.0, 90.0, 0.0 };
var zero = new[] { 0.0, 0.0, 0.0, 0.0, 0.0, 0.0 };

double[] tcp = await robot.AposToCposPose(jp, zero, zero);
Console.WriteLine($"正运动学结果: [{string.Join(", ", tcp)}]");

try
{
    double[] joints = await robot.CposToAposJoints(tcp, jp);
    Console.WriteLine($"逆运动学结果: [{string.Join(", ", joints)}]");
}
catch (InvalidOperationException)
{
    Console.WriteLine("逆运动学: 给定参考下无解。");
}

double[] offsetResult = await robot.CalculateRelativePoseResult(
    tcp, new[] { 0, 0, -100, 0, 0, 0 }, RelativePoseCoorType.Tool);
Console.WriteLine($"相对位姿: [{string.Join(", ", offsetResult)}]");

sub.Dispose();
}
finally
{
    robot.Disconnect();
}

```

.NET Framework 4.6.2 特别说明

本文档涵盖在 .NET Framework 4.6.2 上使用 Codroid SDK 时的平台限制、定时行为、兼容层和构建注意事项。

1. 平台限制

.NET Framework 4.6.2 仅在 **Windows** 上运行。不支持 Linux 和 macOS。

```
<!-- 仅在 Windows 上有效 -->
<TargetFramework>net462</TargetFramework>
```

2. 目标框架

SDK 采用多目标构建。net462 目标与 net6.0 和 net8.0 一同构建。

```
<!-- 摘自 CodroidCS.csproj -->
<TargetFrameworks>net462;net6.0;net8.0</TargetFrameworks>
```

你的测试项目应引用 SDK 并以 net462 为目标：

```
<PropertyGroup>
  <OutputType>Exe</OutputType>
  <TargetFramework>net462</TargetFramework>
  <LangVersion>10</LangVersion>
</PropertyGroup>

<ItemGroup>
  <ProjectReference Include="..\CodroidSDK\CodroidCS.csproj" />
</ItemGroup>
```

3. CRI 250Hz 服务等级

默认且受支持的 CRI 实时控制频率为 **250Hz** (periodMs = 4)。

- periodMs = 4 是**默认服务等级** -- 即开即用、经过测试。
- periodMs != 4 (包括 500Hz / 1000Hz) **不在**默认服务等级内。更高频率需要**现场验证**抖动和控制器行为。

4. 定时实现

在 .NET 6+ 上, SendTrajectory 使用 PeriodicTimer。在 net462 上, PeriodicTimer 不可用。SDK 回退到：

- Stopwatch -- 高分辨率耗时测量。
- Thread.Sleep(1) -- 剩余时间 > 1.5ms 时使用 (让出 CPU)。
- Thread.SpinWait(50) -- 剩余时间 <= 1.5ms 时使用 (忙等待以保精度)。

```
// 简化的 net462 定时循环
long ticksPerPeriod = (long)(Stopwatch.Frequency * periodMs / 1000.0);
long nextTick = stopwatch.ElapsedTicks;

foreach (var point in trajectory)
{
    await SendCommand(point.Position, space, ct);
    nextTick += ticksPerPeriod;
    long remainingTicks = nextTick - stopwatch.ElapsedTicks;

    if (remainingTicks > 0)
    {
        double remainingMs = remainingTicks * 1000.0 / Stopwatch.Frequency;
        if (remainingMs > 1.5)
            Thread.Sleep(1);
        else
            Thread.SpinWait(50);
    }
}
```

5. 抖动统计

在 net462 上 `SendTrajectory` 完成后，SDK 通过 `Trace.TraceInformation` 输出抖动统计。

```
[Codroidsdk][net462] CRI SendTrajectory statistics:
Duration: 4.12s
Frames sent: 1031
Average period: 4.005ms
Max period: 5.823ms
Overruns (>6ms): 0
Max consecutive overruns: 0
UDP exceptions: 0
```

指标	说明
Duration	总耗时
Frames sent	发送的 UDP 帧数
Average period	平均帧间隔
Max period	最大帧间隔
Overruns (>6ms)	超过 6ms 间隔的帧数
Max consecutive overruns	最长连续超限次数
UDP exceptions	发送期间的 Socket 异常

6. 兼容层

五个兼容文件在目标为 net462 时提供 .NET 6+ 缺失的 API。全部位于 `CodroidSDK/Compat/`。

6.1 ArgumentNullException.ThrowIfNull

文件: `Polyfills.cs`

在 net462 上, `ArgumentNullException.ThrowIfNull` 不存在。SDK 提供 `Polyfills.ThrowIfNull`。

```
Polyfills.ThrowIfNull(argument); // 自动捕获参数名
```

6.2 Math.Clamp

文件: `MathPolyfills.cs`

`Math.Clamp` 在 .NET Framework 中不可用。SDK 提供 `MathPolyfills.Clamp`。

```
int clamped = MathPolyfills.Clamp(value, min, max);
double clampedD = MathPolyfills.Clamp(value, 0.0, 1.0);
```

6.3 double.IsFinite

文件: `DoublePolyfills.cs`

```
bool ok = DoublePolyfills.IsFinite(d);
```

6.4 IsExternalInit

文件: `IsExternalInit.cs`

`init` 访问器关键字需要的标记类型, 在 net462 中不存在。此兼容文件添加该类型。

```
public string Name { get; init; } = "";
```

6.5 CallerArgumentExpressionAttribute

文件: `CallerArgumentExpressionAttribute.cs`

`[CallerArgumentExpression]` 属性是 C# 10 / .NET 6+ 特性。此兼容文件在 net462 上声明该属性。

7. NuGet 依赖

当目标为 net462 时, SDK 引入两个额外的 NuGet 包:

包	版本	用途
<code>System.Text.Json</code>	8.0.5	JSON 序列化
<code>System.Memory</code>	4.5.5	<code>Span<T></code> 、 <code>Memory<T></code> 支持

8. UdpClient 差异

`UdpClient` API 在 .NET Framework 和 .NET 6+ 之间存在差异。SDK 使用条件编译处理。

SendAsync

在 net462 上, `UdpClient.SendAsync` 不支持 `ReadOnlyMemory<byte>`。SDK 改用同步 `Send` 方法。

```
#if NET462
    _udp.Send(buffer, length, target);
    await Task.CompletedTask;
#else
    await _udp.SendAsync(buffer.AsMemory(), target, ct);
#endif
```

ReceiveAsync with CancellationToken

在 net462 上, `ReceiveAsync(CancellationToken)` 不可用。SDK 注册取消回调来关闭套接字。

```
#if NET462
    using var reg = token.Register(() =>
    {
        try { _udpClient?.Close(); } catch { }
    });
#else
    await _udpClient.ReceiveAsync(token);
#endif
```

WriteDoubleLittleEndian

在 net462 上, `BinaryPrimitives.WriteDoubleLittleEndian(Span<byte>, double)` 不可用。SDK 回退到 `BitConverter.GetBytes` 并手动处理字节序。

```
#if NET462
    byte[] bytes = BitConverter.GetBytes(value);
    if (!BitConverter.IsLittleEndian) Array.Reverse(bytes);
    Array.Copy(bytes, 0, buffer, offset, 8);
#else
    BinaryPrimitives.WriteDoubleLittleEndian(buffer.AsSpan(offset, 8), value);
#endif
```

9. LangVersion = 10

SDK 和 net462 测试项目均设置 `<LangVersion>10</LangVersion>`。这使得即使在 net462 上也能使用 C# 10 特性。

```
<PropertyGroup>
  <TargetFramework>net462</TargetFramework>
  <LangVersion>10</LangVersion>
  <ImplicitUsings>enable</ImplicitUsings>
  <Nullable>enable</Nullable>
</PropertyGroup>
```

10. 运行 net462 测试项目

基本 API 测试

```
# 完整套件
dotnet run --project CodroidTestNet462/CodroidTestNet462.csproj -- 192.168.8.10

# 单项 IO 测试
dotnet run --project CodroidTestNet462/CodroidTestNet462.csproj -- io 192.168.8.10

# 单项寄存器测试
dotnet run --project CodroidTestNet462/CodroidTestNet462.csproj -- register 192.168.8.10
```

CRI 实时控制测试

```
# 全部段（关节 + 笛卡尔 + 路径）
dotnet run --project CodroidCRITestNet462/CodroidCRITestNet462.csproj

# 仅关节
dotnet run --project CodroidCRITestNet462/CodroidCRITestNet462.csproj -- joint

# 自定义速度的笛卡尔
dotnet run --project CodroidCRITestNet462/CodroidCRITestNet462.csproj -- cart --speed 120 --
accel 600

# 时长模式（6 秒）
dotnet run --project CodroidCRITestNet462/CodroidCRITestNet462.csproj -- cart --duration 6
```

net462 与 net6.0+ 差异总结

方面	net462	net6.0+
平台	仅 Windows	Windows、Linux、 macOS
CRI 定时器	Stopwatch + Thread.Sleep(1) + SpinWait(50)	PeriodicTimer
UDP 发送	UdpClient.Send（同步）	UdpClient.SendAsync（异步）
UDP 接收取消	token.Register(Close)	ReceiveAsync(token)
double 写入	BitConverter.GetBytes + 手动字节序	BinaryPrimitives.WriteDoubleLittleEndian
Math.Clamp	MathPolyfills.Clamp	Math.Clamp
double.IsFinite	DoublePolyfills.IsFinite	double.IsFinite

方面	net462	net6.0+
<code>init</code> 访问器	兼容 (<code>IsExternalInit.cs</code>)	内置
<code>[CallerArgumentExpression]</code>	兼容	内置
额外 NuGet	<code>System.Text.Json 8.0.5</code> 、 <code>System.Memory 4.5.5</code>	无